

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_0febe894-ff9\agent-aa7383e730aa4c4ed.jsonl`  
- SHA-256: `b8ee7136af56a9f4a4c8fde107c6088038e188ce9481fd095be3b5fb280d5fee`  
- Source modified: `2026-07-07T03:37:08+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_0febe894-ff9`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-07T03:35:16.487Z)

Reviewing GitHub PR #58 "docs: refresh current project documentation" (branch agent/docs-sweep -> main). DOCS-ONLY refresh + a few doc-accuracy source edits (help text, docstrings, Dockerfile comment, pyproject description). +782/-787, 14 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr58. This is an ACCURACY review: the docs must match the MERGED code (Tracks B-F + ADR 0012 F1-F7 are all merged). Read the doc diff:

```
git -C C:/Users/avidu/Projects/Telegram diff 7fadffb..origin/agent/docs-sweep -- <file>  
and cross-check every concrete claim against the actual code (src/tg_video_filter/*.py,  
docs/adr/*). Read code via Read/Grep; run  
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check defaults/behavior.
```

KEY GROUND TRUTH (verified by the main reviewer) to check the docs against:

- Selective scanning is OPT-IN: new sources default watched=false (should_ingest_source); the watched flag is user-controlled (set_source_watched / /watch,/unwatch);
upsert_source no longer clobbers watched (a fix in #41). Commands: /channels, /watch <id|@name>, /unwatch, /watchlist.
- FilterConfig exclusion DEFAULTS: exclude_gif=True, exclude_round=True, exclude_nosound=False (i.e. GIF/round excluded, nosound allowed). The old "all off by default" was WRONG.
- Policy pipeline stages (evaluate_pipeline order): topic-preferences -> url-reputation -> caption-safety -> deep-inspection. Deep inspection is a budget BOUNDARY only; NO production fetcher is wired (ctx.fetcher/budget are None in the live path), so enabling deep_inspect drops items.
- Config lives in policies.config_json (raw dict); save_filter_config MERGES flat FilterConfig keys so ADR 0012 keys (topics/security_checks/deep_inspect) survive a /set. There is NO /set command to set those policy keys yet (dormant).
- Tables: accounts, telegram_sessions, sources, content_items, policies, destinations, routing_rules, delivery_attempts, policy_decisions (+ *_deprecated). policy_decisions has NO TTL/retention cleanup (deferred, issue #56).
- Dedup: 3-tier (message instance, Telegram document.id, metadata fingerprint) via content_items.dedup_key. Delivery modes forward/link/both to me/@username/chat id.
- /why <content_item_id> is account-scoped (get_content_item_for_account), renders the stored policy trace or a caveated metadata replay; markdown-escaped.

Ground rules: READ the doc diff AND the code, cite file:line. Report ONLY doc claims in THIS PR diff that are FACTUALLY WRONG or MISLEADING vs the merged code (a stale table name, a command that does not exist, a wrong default, a wrong ADR status, a "does X" the code does not do). No style nitpicks unless misleading. Severity: high (a wrong claim a user/operator would act on and be broken by, e.g. wrong command/flag/deploy step), medium (misleading/ambiguous/stale), low (typo/broken link/minor).

DIMENSION: verify README.md and docs/design.md against the code. Check: feature list, command surface, table/schema names, dedup description, delivery modes, policy-stage description and order, the "what is not built" list (does it correctly include the un-wired deep-inspection fetcher and the missing retention cleanup?), the opt-in scanning description (watched=false

default), and the data-model/architecture claims. Flag any claim the code does not support.

2. assistant (2026-07-07T03:35:19.027Z)

I'll review PR #58 for documentation accuracy. Let me start by examining the doc diff and the actual code.

3. user (2026-07-07T03:35:20.991Z)

```
Dockerfile | 3 +-
README.md | 195 ++++++---
docs/adr/0007-inline-config-commands.md | 9 +
docs/adr/0009-selective-channel-scanning.md | 10 +
docs/adr/0010-identity-and-session-separation.md | 9 +-
docs/adr/0011-provider-neutral-domain-schema.md | 22 +-
docs/adr/0012-policy-pipeline.md | 70 ++--
docs/design.md | 473 ++++++-----
docs/plan.md | 334 +++++-----
docs/runbook.md | 416 ++++++-----
pyproject.toml | 2 +-
src/tg_video_filter/command_service.py | 4 +-
src/tg_video_filter/models.py | 20 +-
tests/test_commands.py | 2 +-
14 files changed, 782 insertions(+), 787 deletions(-)
```

4. assistant (2026-07-07T03:35:23.170Z)

Let me read the doc diffs and the relevant code in parallel.

5. user (2026-07-07T03:35:25.846Z)

```
diff --git a/README.md b/README.md
index 80de756..d4c6ecc 100644
--- a/README.md
+++ b/README.md
@@ -1,75 +1,148 @@
# telegram-video-filter
```

-A Telethon-based **userbot** that logs in as your own Telegram account, watches new video messages across all the channels/groups you subscribe to, applies configurable filters (duration, resolution, file size), and **forwards matching videos** to your Saved Messages in real time.

-> ?? This project automates a **user** account (a "userbot") via MTProto. Keep volumes reasonable and respect Telegram's Terms of Service. You are responsible for your own account. See docs/adr/0001-userbot-over-botapi.md.

Status

-? **v1 complete** ? all 6 build phases merged via PR. The app runs end-to-end: bootstrap a user account ? manager watches their channels ? filters videos ? forwards matches to Saved Messages. See docs/plan.md for the build history and docs/runbook.md to deploy. phased build plan and current status.

What it does (v1)

- Watches every channel/group your account is a member of ? no per-channel setup.
- Reads video metadata **without downloading** the file (duration, width/height, size, MIME).
- Applies an **AND** of your filters, e.g.:
 - duration between 5s and 2h,
 - height ? 1080 (1080p or better),

- size ? 2 GiB.
- Forwards each match to your ****Saved Messages**** in real time, or to a
- ****channel you own**** if you set ``delivery_target`` (per-user).
- ****Never forwards the same video twice**** ? three-tier content dedup
- (message instance ? Telegram ``document.id`` ? metadata fingerprint) means
- a video cross-posted to 5 channels is delivered once. See
- [``docs/adr/0006-content-dedup.md``](docs/adr/0006-content-dedup.md).
- Supports multiple users (<100) on a single host, each with its own session,
- filters, delivery mode/target, and dedup state.
-
- ## What it does NOT do yet (v2+)
-
- Public-channel ``/add @channel`` join, manual-add ingestion.
- Scheduled digest delivery and on-demand ``/list`` / ``/search``.
- ffproube-based deep inspection (``/inspect``).
- Codec / keyword / MIME filters.
- +A Telethon-based Telegram ****userbot**** that logs in as your own account, filters
- +video messages from selected sources, and delivers matches to Saved Messages or
- +another Telegram destination you control.
- +
- +The app is metadata-first: it reads Telegram message metadata and captions
- +without downloading media for the normal path. It is built for a small
- +multi-user deployment on one host, with SQLite state and file-backed Telethon
- +sessions under ``data/``.
- +
- +> This project automates a ****user**** account via MTProto. Keep volumes
- +> reasonable and respect Telegram's Terms of Service. Session files are
- +> credentials. See [ADR 0001](docs/adr/0001-userbot-over-botapi.md).
- +
- +## Current Status
- +
- +The main userbot path is implemented end to end:
- +
- + Bootstrap one or more Telegram user sessions.
- + Discover Telegram channels/groups as sources.
- + Opt in live ingestion per source with ``/watch``.
- + Filter video metadata with duration, resolution, size, GIF/round/nosound
- + exclusions, and the ADR 0012 policy pipeline.
- + Deliver accepted videos by forward, link, or both.
- + Persist content, delivery state, policy decisions, and explanation traces.
- + Explain historical decisions with ``/why <content_item_id>``.
- +
- +The original v1 plan and the ADR 0012 F1-F7 policy pipeline are complete. The
- +current follow-up is policy-decision retention cleanup; see issue #56.
- +
- +## Core Features
- +
- + ****Selective live scanning****: unknown sources are discovered as ``watched=false``.
- + Use ``/channels``, ``/watch <id|@name>``, ``/unwatch <id|@name>``, and ``/watchlist``
- + from Saved Messages to control live ingestion.
- + ****Metadata filters****: duration bounds, minimum height, maximum size, GIF,
- + round-video, and nosound exclusions.
- + ****Policy stages****: topic preferences, captured-caption safety checks,
- + captured URL/domain reputation checks, and an opt-in deep-inspection boundary.
- + Deep inspection is budgeted but no production fetcher is wired by default.
- + ****Deduplication****: provider-neutral dedup keys catch same message, same
- + Telegram ``document.id``, and same metadata fingerprint.
- + ****Delivery modes****: ``forward``, ``link``, or ``both``, targeting ``"me"``,
- + ``@username``, or a numeric Telegram chat id.
- + ****Backfill****: ``/backfill [N][h|d|w]`` scans historical channel/group messages.
- + Backfill is intentionally broad and should be run carefully on large accounts.
- + ****Explainability****: ``/why <content_item_id>`` renders the stored policy trace
- + when available, or a clearly caveated current metadata replay for older
- + items without a policy-decision row.
- +

```

+## What Is Not Built
+
+- Bot API onboarding/front-door commands from ADR 0008.
+- Public-channel auto-join or manual `/add @channel` ingestion.
+- Scheduled digests and on-demand search/list commands.
+- A production deep-inspection fetcher/downloader.
+- Policy-decision retention cleanup.
+- Horizontal scaling across multiple hosts.
+
+## Quick Start

-## Documentation
+```bash
+cp .env.example .env
+# edit .env with TG_API_ID and TG_API_HASH from https://my.telegram.org
+
+docker compose up -d --build
+python -m tg_video_filter.bootstrap
+docker compose restart app
+```
+
+After startup, send commands to your own Saved Messages:
+
+```text
+/channels
+/watch <source-id-or-title>
+/show config
+/show commands
+```
+
+See [docs/runbook.md](docs/runbook.md) for the full operational workflow.

-- [docs/plan.md](docs/plan.md) ? phased build plan, scope, verification.
-- [docs/design.md](docs/design.md) ? architecture, data model, control flow.
-- [docs/runbook.md](docs/runbook.md) ? deploy, onboard users, rotate
  sessions, handle flood waits.
-- [docs/adr/](docs/adr/) ? Architecture Decision Records:
  - [0001 ? Userbot over Bot API](docs/adr/0001-userbot-over-botapi.md)
  - [0002 ? SQLite over PostgreSQL](docs/adr/0002-sqlite-over-postgres.md)
  - [0003 ? Metadata-first probing](docs/adr/0003-metadata-first-probing.md)
  - [0004 ? Pluggable delivery modes](docs/adr/0004-pluggable-delivery-modes.md)
  - [0005 ? Per-user delivery target](docs/adr/0005-per-user-delivery-target.md)
+## Useful Commands

-## Quick start
+Filter and config:

-See the [runbook](docs/runbook.md) for the full guide. In short:
+- `/show config`
+- `/show commands`
+- `/set size <N><unit>` where unit is `KB`, `MB`, `GB`, or `TB`
+- `/set length <N><unit>` or `/set max length <N><unit>`
+- `/set min length <N><unit>`
+- `/set min resolution <N>p` or `/set min res <N>p`
+- `/set exclude gif|round|nosound on|off`
+
+Source scanning:
+
+- `/channels`
+- `/watch <id|@name|title-fragment>`
+- `/unwatch <id|@name|title-fragment>`
+- `/watchlist`
+
+Backfill and explainability:
+

```

```

+- `/backfill`, `/backfill 8h`, `/backfill 7d`, `/backfill 2w`
+- `/stop`
+- `/why <content_item_id>`
+
+### Documentation
+
+- [docs/runbook.md](docs/runbook.md) - deployment, onboarding, commands,
+ operations, and troubleshooting.
+- [docs/design.md](docs/design.md) - current runtime architecture, data model,
+ control flow, and policy pipeline.
+- [docs/plan.md](docs/plan.md) - implementation status and completed tracks.
+- [docs/adr/](docs/adr/) - decision history:
+ - [0001 - Userbot over Bot API](docs/adr/0001-userbot-over-botapi.md)
+ - [0002 - SQLite over PostgreSQL](docs/adr/0002-sqlite-over-postgres.md)
+ - [0003 - Metadata-first probing](docs/adr/0003-metadata-first-probing.md)
+ - [0004 - Pluggable delivery modes](docs/adr/0004-pluggable-delivery-modes.md)
+ - [0005 - Per-user delivery target](docs/adr/0005-per-user-delivery-target.md)
+ - [0006 - Content dedup](docs/adr/0006-content-dedup.md)
+ - [0007 - Inline config commands](docs/adr/0007-inline-config-commands.md)
+ - [0008 - Bot API onboarding front door](docs/adr/0008-bot-api-onboarding-front-door.md)
+ - [0009 - Selective channel scanning](docs/adr/0009-selective-channel-scanning.md)
+ - [0010 - Identity and session separation](docs/adr/0010-identity-and-session-separation.md)
+ - [0011 - Provider-neutral domain schema](docs/adr/0011-provider-neutral-domain-schema.md)
+ - [0012 - Policy pipeline](docs/adr/0012-policy-pipeline.md)
+
+### Development

    ```bash
-cp .env.example .env # set TG_API_ID, TG_API_HASH from my.telegram.org
-docker compose up -d --build
-python -m tg_video_filter.bootstrap # interactive one-time login per user
-docker compose restart app
+python -m venv .venv
+.\.venv\Scripts\python.exe -m pip install -e .[dev]
+.\.venv\Scripts\python.exe -m ruff check src tests
+.\.venv\Scripts\python.exe -m pytest
    ```

-## Tech stack
+The test suite uses an 80% coverage gate. Live Telethon glue and interactive
+entrypoints are marked `pragma: no cover` and covered by focused pure-unit
+tests around their helpers.
+
+### Tech Stack

-Python 3.11+ · [Telethon](https://docs.telethon.dev/) · aiosqlite (SQLite) ·
-pydantic / pydantic-settings · Docker · pytest · ruff.
+Python 3.11+ · Telethon · aiosqlite/SQLite · pydantic/pydantic-settings ·
+Docker · pytest · ruff.

## License

```

6. user (2026-07-07T03:35:26.476Z)

```

diff --git a/docs/design.md b/docs/design.md
index 45fc4aa..1c112c4 100644
--- a/docs/design.md
+++ b/docs/design.md
@@ -1,280 +1,233 @@
-# Design: Telegram Video Filter Userbot (v1)
-
-This document describes the runtime architecture, data model, control flow, and
-key technical contracts for v1. See [plan.md](./plan.md) for the phased build
-order and [adr/](./adr/) for the rationale behind the major decisions.
-

```

[illegible]

```

-
-?
-????????????????????????????????????????????????????????????
-? SQLite (aiosqlite) db.py ?
-? tables: users, channels, videos, filter_config ?
-? repos: get/create user, upsert video (dedup), ?
-? load/save filter_config ?
-????????????????????????????????????????????????????????????
+
+## Design: Telegram Video Filter Userbot
+
+This document describes the current runtime architecture and data contracts.
+Decision rationale lives in [docs/adr/](./adr/); operational steps live in
+[runbook.md](./runbook.md).
+
+## 1. Goals
+
+- Run one Telethon MTPROTO userbot session per enabled Telegram account.
+- Discover Telegram channels/groups as provider-neutral `sources`.
+- Ingest live videos only from sources explicitly marked `watched=true`.
+- Extract message metadata and captions without downloading media on the normal
+ path.
+- Apply metadata filters and the ADR 0012 policy pipeline.
+- Deliver accepted content to a configured Telegram destination.
+- Persist enough state to deduplicate, retry/inspect delivery state, and explain
+ decisions with `/why`.
+
+## 2. Non-goals
+
+- Bot API onboarding/front-door commands. ADR 0008 defines the shape, but it is
+ not implemented.
+- Public-channel auto-join or manual `/add @channel` ingestion.
+- Scheduled digest delivery or search/list commands.
+- Production deep-inspection fetch/download work. The policy stage and budget
+ boundary exist; no fetcher is wired by default.
+- Horizontal scaling across multiple hosts.
+
+## 3. Runtime Components
+
+```text
+python -m tg_video_filter
+|
+ v
+manager.run()
+|
+ +-- loads runnable telegram_sessions
+ +-- starts one Telethon client task per session
+|
+ +-- live NewMessage handler
+|   source watch gate -> video metadata -> dedup -> FilterEngine
+|   -> PolicyStage pipeline -> content_items/policy_decisions
+|   -> delivery.deliver()
+|
+ +-- Saved Messages command handlers
+|   /show, /set, /channels, /watch, /unwatch, /watchlist, /why
+|   /backfill, /stop
+```
+
+## 3. Data model
+
+-
+-All tables live in a single SQLite file (default `data/app.db`).
+-
+### `users`
+
+| column | type | notes |
+|---|---|---|
+| `id` | INTEGER PK | Telegram user id (the account the bot logs in as) |
+| `phone` | TEXT | E.164 phone, used only for the bootstrap session |

```

```

-| `enabled` | INTEGER (0/1) | If 0, manager skips this user |
-| `created_at` | TEXT (ISO8601) | |
-
-### `channels`
-Row per (user, channel) the user is a member of. Populated lazily as messages
-arrive; primarily informational in v1.
-
-| column | type | notes |
-|---|---|---|
-| `id` | INTEGER PK | surrogate |
-| `user_id` | INTEGER FK ? users.id | |
-| `channel_id` | INTEGER | Telegram channel/chat id |
-| `title` | TEXT | last seen title |
-| `seen_first_at` | TEXT (ISO8601) | |
-
-Unique index on `(user_id, channel_id)`.
-
-### `videos`
-Seen-state + metadata log. Drives dedup and future `/search`.
-
-| column | type | notes |
-|---|---|---|
-| `id` | INTEGER PK | surrogate |
-| `user_id` | INTEGER FK ? users.id | |
-| `channel_id` | INTEGER | origin channel |
-| `message_id` | INTEGER | Telegram message id within the channel |
-| `document_id` | INTEGER | Telegram `document.id` ? reused across forwards; **Tier-1 content
dedup** (nullable; ADR 0006) |
-| `duration_s` | INTEGER | from `DocumentAttributeVideo.duration` |
-| `width` | INTEGER | from `DocumentAttributeVideo.w` |
-| `height` | INTEGER | from `DocumentAttributeVideo.h` |
-| `size_bytes` | INTEGER | from `document.size` |
-| `mime_type` | TEXT | from `document.mime_type` |
-| `matched` | INTEGER (0/1) | did it pass filters |
-| `forwarded` | INTEGER (0/1) | did the forward succeed |
-| `seen_at` | TEXT (ISO8601) | |
-
-Unique index on `(user_id, channel_id, message_id)` ? the Tier-0 dedup key.
-
-A partial unique index on `(user_id, document_id) WHERE document_id IS NOT NULL`
-enforces **Tier-1 content dedup** (same Telegram file forwarded across channels).
-**Tier-2 content dedup** uses a metadata fingerprint
-`(size_bytes, duration_s, height, width)` lookup with no DB constraint (it is
-heuristic, not cryptographic). See [ADR 0006](./adr/0006-content-dedup.md).
-
-### `filter_config`
-One row per user (v1). Stored as JSON blob for forward-compat as new filters land.
-
-| column | type | notes |
-|---|---|---|
-| `user_id` | INTEGER PK FK ? users.id | |
-| `config_json` | TEXT | serialized `FilterConfig` |
-
-Default v1 config (v1.1 adds ``delivery_mode``):
+The process uses one SQLite database. Telethon session files live under
+`SESSIONS_DIR` (`data/sessions` locally, `/data/sessions` in Docker).
+
+### 4. Identity and Sessions
+
+ADR 0010 split identity from MTPROTO sessions:
+
+- `accounts` owns config, sources, content, policies, destinations, and routes.
+- `telegram_sessions` is optional MTPROTO state for accounts that run as
+ userbots.
+- Legacy `users` APIs still work as shims over `accounts` +

```



```

+ `telegram_sessions`.
+
+This lets a future Bot API front door create accounts without also creating a
+Telethon session.
+
+### 5. Source Scanning
+
+Live scanning is opt-in:
+
+1. A first live message from an unknown Telegram channel/group creates a
+   `sources` row with `watched=false`.
+2. The live handler returns immediately for unwatched sources.
+3. Users list and toggle sources from Saved Messages:
+   - `/channels`
+   - `/watch <id|@name|title-fragment>`
+   - `/unwatch <id|@name|title-fragment>`
+   - `/watchlist`
+
+Backfill is different: `/backfill` iterates historical dialogs directly and can
+scan broadly. It should be treated as an operational tool, not the live watch
+gate.
+
+### 6. Data Model
+
+Important tables:
+
+| Table | Purpose |
+|---|---|
+| `accounts` | Provider-neutral owner identity. |
+| `telegram_sessions` | Optional Telethon session metadata per account. |
+| `sources` | Provider-neutral ingest sources, including `watched`. |
+| `content_items` | Normalized content records and dedup keys. |
+| `policies` | JSON policy config; one default policy per account. |
+| `destinations` | Delivery targets such as `"me"`, `@channel`, or chat id. |
+| `routing_rules` | Source -> policy -> destination mapping skeleton. |
+| `delivery_attempts` | Per-content/per-destination delivery state. |
+| `policy_decisions` | Persisted ADR 0012 pass/fail decision and stage trace. |
+
+Legacy tables/models (`users`, `channels`, `videos`, `filter_config`,
+`VideoRecord`, `FilterConfig`) are still supported through compatibility shims.
+New code should prefer `Account`, `TelegramSession`, `Source`, `ContentItem`,
+`Policy`, `Destination`, `DeliveryAttempt`, and `PolicyDecision`.
+
+#### Dedup
+
+`content_items.dedup_key` is provider-prefixed and unique per account. Telegram
+dedup is computed cheapest-first:
+
+1. Same source/message instance.
+2. Same Telegram `document.id`.
+3. Same metadata fingerprint `(size_bytes, duration_s, height, width)`.
+
+### 7. Filter and Policy Evaluation
+
+Evaluation is two-layered:
+
+1. `FilterEngine` applies the flat metadata `FilterConfig` rules:
+   - min/max duration
+   - minimum height
+   - maximum size
+   - GIF, round-video, and nosound exclusions
+2. If metadata filters pass and a `Source` exists, the ADR 0012 policy pipeline
+   evaluates:
+   - `TopicPreferenceStage`
+   - `UrlReputationStage`

```

```

+   - `CaptionSafetyStage`
+   - `DeepInspectionStage`
+
+The policy stages return ordered `StageResult` values. A failed stage
+short-circuits the pipeline. F6 persists the final pass/fail and full stage
+trace in `policy_decisions`. F7 exposes it through `/why <content_item_id>`.
+
+### Policy Config Shape
+
+Flat filter fields still live in `FilterConfig`, but `policies.config_json` can
+also hold future policy keys. The DB shims merge flat updates into the existing
+JSON object so `/set` commands do not erase unknown policy keys.
+
+Relevant policy keys today:
+
+```json
+{
+  "min_duration_s": 5,
+  "max_duration_s": 7200,
+  "target_height": 1080,
+  "max_size_mb": 2048,
+  "delivery_mode": "forward"
+  "topics": {
+    "include": ["ai"],
+    "exclude": ["crypto"],
+    "mode": "any"
+  },
+  "security_checks": {
+    "blocked_domains": ["bad.example"],
+    "unsafe_caption_terms": ["malware"],
+    "unsafe_caption_patterns": ["free\\s+money"]
+  },
+  "deep_inspect": false
+}
+```

-## 4. Delivery contract (v1.1)
+There are no user-facing `/set` commands for these policy keys yet; edit
+`policies.config_json` deliberately if you need them.

-```python
-class DeliveryMode(str, Enum):
-    FORWARD = "forward" # forward the video itself (default)
-    LINK = "link" # send a t.me link + metadata summary
-    BOTH = "both" # forward + send link
+## 8. Delivery

-async def deliver(client, message, conn, video_id, mode, *, link_text=None) -> bool
-```

+`FilterConfig.delivery_mode` controls how accepted content is delivered:

-The `deliver` dispatcher in `delivery.py` routes to:
-- `FORWARD` ? `client.forward_messages("me", message)`
-- `LINK` ? `client.send_message("me", link_text)` where `link_text` is
-  built by the pure `build_link_text(...)` function
-- `BOTH` ? forward + send link
+| Mode | Behavior |
+|---|---|
+| `forward` | Forward the Telegram message to the target. |
+| `link` | Send a `t.me` link plus metadata summary. Falls back to forward if no link text is
+available. |
+| `both` | Forward and send the link summary. |

-Link message format:
-```

```

```

-? 1080p · 5min · 480MB
-#ChannelName
-https://t.me/c/1234567890/42
-```
+`FilterConfig.delivery_target` is passed to Telethon as the destination entity:

-If ``link_text`` is ``None`` in LINK/BOTH mode (e.g. ``message.link`` was
-unavailable), falls back to FORWARD so no match is silently dropped. See
-[ADR 0004](./adr/0004-pluggable-delivery-modes.md).
+- ``"me"`` for Saved Messages
+- ``@username``
+- numeric chat id string such as ``-1001234567890``

-### 5. Filter engine contract
+Delivery writes `delivery_attempts`; policy rejection is not a delivery failure
+and is stored in `policy_decisions` instead.

-```python
-class VideoMeta:
-    # extracted from the Telethon message
-    duration_s: int | None
-    width: int | None
-    height: int | None
-    size_bytes: int | None
-    mime_type: str | None
+### 9. Commands

-class FilterResult:
-    matched: bool
-    reasons: list[str]
-    # human-readable, one per evaluated rule
+Saved Messages commands are handled by provider-independent `CommandService`
+plus a thin Telethon adapter.

-class Filter(ABC):
-    # filters/base.py
-    @abstractmethod
-    def evaluate(self, meta: VideoMeta) -> FilterResult: ...
+Config:

-class FilterEngine:
-    # filters/__init__.py
-    def __init__(self, cfg: FilterConfig): ...
-    def evaluate(self, meta: VideoMeta) -> FilterResult: ... # AND of all rules
-```
+- ``/show config``
+- ``/show commands``
+- ``/set size <N><unit>``
+- ``/set length <N><unit>`` or ``/set max length <N><unit>``
+- ``/set min length <N><unit>``
+- ``/set min resolution <N>p`` or ``/set min res <N>p``
+- ``/set exclude gif|round|nosound on|off``

-Semantics: AND ? a video is forwarded only if every enabled rule passes.
-A rule that cannot be evaluated (e.g. `duration` is None) is treated as
-not matched, with a reason like "duration unknown".
-
-### v1 rules
-| rule | pass condition | fail reason examples |
-|---|---|---|
-| `DurationFilter` | `min_duration_s <= duration <= max_duration_s` | `"duration 3s < min 5s"`,
-| | | | |
-| | | | |
-| `ResolutionFilter` | `height >= target_height` | `"height 720 < target 1080" |
-| `SizeFilter` | `size_bytes <= max_size_mb * 1_048_576` | `"size 3000MiB > max 2048MiB" |
-| | | | |
-
-### 6. Control flow: incoming video
-
-1. Telethon fires `events.NewMessage(incoming=True)`.
-2. Handler checks `event.message.video` is not None; if None, return.

```

```

-3. Build `VideoMeta` from the message document + `DocumentAttributeVideo`.
-4. Run the three-tier dedup check (cheapest first; see ADR 0006):
-   - Tier 0: `(user_id, channel_id, message_id)` ? same message instance.
-   - Tier 1: `(user_id, document_id)` ? same Telegram file forwarded across
-     channels (DB-enforced via partial unique index).
-   - Tier 2: `(size_bytes, duration_s, height, width)` fingerprint ?
-     re-upload of an identical file with a new `document.id`.
-   If any tier matches, return (already processed).
-5. Build `FilterEngine` from the user's `filter_config` (cached for the session).
-6. `result = engine.evaluate(meta)`.
-7. Insert a `videos` row with `matched=result.matched`.
-8. If `result.matched`: call `delivery.deliver(client, message, conn, id,
-   mode, link_text=..., target=cfg.delivery_target)`. `delivery_target` is a
-   Telethon entity: `"me"` (Saved Messages, default), `"@username"`, or a
-   numeric chat id string (see ADR 0005). On success set `forwarded=1`. On
-   `FloodWaitError`, sleep and retry once; on second failure leave
-   `forwarded=0` and log.
-
-## 7. Control flow: manager lifecycle
+Source scanning:
-
-```
-main()
- ?? load Settings (env)
- ?? await db.init(db_path)           # create tables if missing
- ?? users = await db.list_enabled_users()
- ?? for u in users:
- ?     task = asyncio.create_task(run_user(u))
- ?? install SIGTERM/SIGINT handlers ? cancel all tasks
- ?? await asyncio.gather(*tasks)
-
-run_user(u)
- ?? client = await build_client(u)    # SQLiteSession at data/sessions/user_{id}.session
- ?? engine = FilterEngine(await db.load_filter_config(u.id))
- ?? register NewMessage handler bound to (client, u, engine)
- ?? await client.run_until_disconnected()
-```
+- `/channels`
+- `/watch <id|@name|title-fragment>`
+- `/unwatch <id|@name|title-fragment>`
+- `/watchlist`
+
+Backfill and explainability:
+
+- `/backfill [N][h|d|w]`
+- `/stop`
+- `/why <content_item_id>`
+
+`/why` is account-scoped before reading decisions, escapes untrusted Markdown
+values, breaks raw autolinks, and caps replies below Telegram's message limit.
+
+## 10. Observability
+
+The app logs to stdout. Important events include:
+
+- `manager_started`
+- `client_connected`
+- `video_seen`
+- `video_forwarded`
+- `delivery_record_failed`
+- `flood_wait`
+- `config_updated`
+- policy-stage warnings for invalid config or failed deep inspection
+- `client_disconnected`
+

```

+There is no metrics endpoint. Docker restart policy and logs are the current operational surface.

+
+## 11. Security Notes

-## 8. Security & operational notes

-
-- ****Secrets****: `TG\_API\_ID`, `TG\_API\_HASH`, and session files (`\*.session`) are sensitive. `.gitignore` excludes `\*.session` and `.env`. In Docker, the `data/` volume holds both the SQLite DB and session files ? back it up, treat it as a credential.
-- ****Session rotation****: see [runbook.md](./runbook.md) `$"Rotate a user session"`.
-- ****Flood control****: forwarding too fast triggers `FloodWaitError`. v1 sleeps the requested duration + small jitter and retries once. If your account gets flagged, slow down or reduce the number of channels.
-- ****ToS****: a userbot automates a *user* account. Keep volume reasonable, don't spam, don't scrape at scale. The owner is responsible for their account.

-## 9. Observability (v1 minimum)

-
-Structured logging to stdout (JSON Lines recommended). Key events:
-- `manager\_started` `{users: N}`
-- `client\_connected` `{user\_id}`
-- `video\_seen` `{user\_id, channel\_id, message\_id, matched, reasons}`
-- `video\_forwarded` `{user\_id, channel\_id, message\_id}`
-- `flood\_wait` `{user\_id, seconds}`
-- `client\_disconnected` `{user\_id, reason}`
-
-A `/health` or simple process-level liveness check is a v2 concern; v1 relies on Docker restart policy.
+- Treat `.env`, `TG\_API\_HASH`, and `\*.session` files as credentials.
+- `/why` only exposes content belonging to the requesting account.
+- Untrusted captions, URLs, reasons, and metadata are escaped in `/why` output.
+- Backfill can generate significant Telegram traffic; run it intentionally.
+- Userbot automation can put the Telegram account at risk if abused.

7. assistant (2026-07-07T03:35:29.034Z)

Now let me read the actual source code to verify claims. Let me look at the key files.

8. user (2026-07-07T03:35:31.138Z)

__init__.py
__main__.py
__pycache__
backfill.py
bootstrap.py
command_service.py
commands.py
config.py
db.py
delivery.py
filters
manager.py
models.py
policy.py
telethon_client.py

9. user (2026-07-07T03:35:32.061Z)

Exit code 2
0001-userbot-over-botapi.md
0002-sqlite-over-postgres.md
0003-metadata-first-probing.md

0004-pluggable-delivery-modes.md
0005-per-user-delivery-target.md
0006-content-dedup.md
0007-inline-config-commands.md
0008-bot-api-onboarding-front-door.md
0009-selective-channel-scanning.md
0010-identity-and-session-separation.md
0011-provider-neutral-domain-schema.md
0012-policy-pipeline.md

10. user (2026-07-07T03:35:34.699Z)

```
1      """Pydantic data models for users, channels, videos, and filter config.
2
3      These models are the in-memory representation used by the repository layer
4      (`db.py`) and the filter engine (`filters/`). They are deliberately
5      decoupled from Telethon's own types so the filter engine stays pure and
6      unit-testable without a live Telegram connection.
7      """
8
9      from __future__ import annotations
10
11     from datetime import UTC, datetime
12     from enum import Enum
13
14     from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
15
16
17     def _utcnow() -> datetime:
18         """Timezone-aware UTC 'now', used as a default factory."""
19         return datetime.now(UTC)
20
21
22     class DeliveryMode(str, Enum):
23         """How a matched video is delivered to the user.
24
25         - ``FORWARD``: forward the video itself to Saved Messages (re-uploads).
26         - ``LINK``: send a text message with a t.me link + metadata summary.
27         - ``BOTH``: forward the video AND send the link message.
28         """
29
30         FORWARD = "forward"
31         LINK = "link"
32         BOTH = "both"
33
34
35     class Account(BaseModel):
36         """A product identity ? the owner of config, channels, routes, and content.
37
38         Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
39         is the bridge used by the Bot API front-door (`from.id`) and by the
40         legacy userbot path; it is nullable so future providers that don't have a
41         Telegram id (e.g. WhatsApp) can still own an ``Account``.
42         """
43
44         model_config = ConfigDict(frozen=True)
45
46         id: int = Field(description="Stable surrogate PK; the owner key for all downstream
47         tables")
48         telegram_id: int | None = Field(
49             default=None,
50             description="Telegram user id (bot from.id OR userbot account id). UNIQUE when
51             set.",
52         )
53         provider: str = Field(default="telegram", description="Provider namespace; future:
```

```

'whatsapp'")
52         display_name: str | None = None
53         created_at: datetime = Field(default_factory=_utcnow)
54
55
56     class TelegramSession(BaseModel):
57         """An optional MProto (Telethon) session linked to an :class:`Account`.
58
59         A bot-only user has an ``Account`` row but no ``TelegramSession`` row;
60         that is what makes them structurally invisible to ``manager.run``
61         (ADR 0010). ``session_path`` is data-driven so the file-location
62         convention is explicit rather than derived from the PK at runtime.
63         """
64
65         model_config = ConfigDict(frozen=True)
66
67         account_id: int = Field(description="FK ? accounts.id; PK (one session per account
in v1)")
68         phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
69         session_path: str = Field(description="File-backed Telethon session path
(user_<id>.session)")
70         enabled: bool = Field(
71             default=True,
72             description="If False, the manager skips this session (not the account)",
73         )
74         created_at: datetime = Field(default_factory=_utcnow)
75         last_login_at: datetime | None = None
76
77
78     class User(BaseModel):
79         """Deprecated: legacy v1 model conflating identity + MProto session.
80
81         Retained as a backwards-compatible view over the new ``accounts`` +
82         ``telegram_sessions`` tables (ADR 0010). New code should use
83         :class:`Account` + :class:`TelegramSession` directly. The ``db`` layer
84         maps ``User`` ? (``Account``, ``TelegramSession``) so existing callers
85         and tests keep working without a flag day.
86         """
87
88         model_config = ConfigDict(frozen=True)
89
90         id: int = Field(description="Telegram user id (== accounts.telegram_id for legacy
rows)")
91         phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
92         enabled: bool = Field(default=True, description="If False, the manager skips this
session")
93         created_at: datetime = Field(default_factory=_utcnow)
94
95
96     class Channel(BaseModel):
97         """A channel/group a user is a member of. Populated lazily as messages arrive.
98
99         Deprecated: a provider-neutral view over :class:`Source` (ADR 0011). New
100         code should use :class:`Source` directly; the ``db`` layer maps
101         ``Channel`` ? ``Source`` so existing callers and tests keep working.
102         """
103
104         model_config = ConfigDict(frozen=True)
105
106         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
107         user_id: int
108         channel_id: int = Field(description="Telegram channel/chat id")
109         title: str | None = None

```

```

110         seen_first_at: datetime = Field(default_factory=_utcnow)
111
112
113     # --- provider-neutral domain models (ADR 0011, Track B) -----
114     #
115     # These generalize the legacy v1 ``Channel`` / ``VideoRecord`` / ``FilterConfig``
116     # shapes so the schema no longer leaks Telegram-specific ids into every future
117     # provider's code path. ``provider`` columns + provider-prefixed ``dedup_key``
118     # make cross-provider collisions structurally impossible. See ADR 0011 for the
119     # full rationale and the two decisions (Option 1A: tiered ``dedup_key``;
120     # Option 2A: ``policies`` generalizes ``filter_config``).
121
122
123     class Source(BaseModel):
124         """A configured place we ingest content from (ADR 0011).
125
126         Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127         providers add their own source kinds). ``provider_source_id`` is the
128         provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129         """
130
131         model_config = ConfigDict(frozen=True)
132
133         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134         account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135         provider: str = Field(default="telegram", description="Provider namespace")
136         provider_source_id: str = Field(
137             description="Provider-native source id (Telegram channel_id, ?)"
138         )
139         title: str | None = None
140         kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141         watched: bool = Field(
142             default=False,
143             description="ADR 0009 selective scanning: ingest from this source iff True",
144         )
145         seen_first_at: datetime = Field(default_factory=_utcnow)
146         last_seen_at: datetime | None = None
147
148
149     class ContentItem(BaseModel):
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. The Telegram path stores videos
153         today (``content_type='video'``) and captures caption text; ``link_url`` is
154         available for future link/article content while URL policy stages currently
155         extract links from captured text at evaluation time.
156         ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
157         wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
158         """
159
160         model_config = ConfigDict(frozen=True)
161
162         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
163         account_id: int = Field(description="FK ? accounts.id")
164         source_id: int = Field(description="FK ? sources.id")
165         provider: str = Field(default="telegram", description="Provider namespace")
166         provider_message_id: str = Field(
167             description="Provider-native message id (Telegram message_id, ?)"
168         )
169         content_type: str = Field(default="video", description="'video' | 'link' | 'article'")
170         text: str | None = Field(default=None, description="Caption / post text (nullable in Phase 1)")
171         link_url: str | None = Field(default=None, description="For link/article posts (Phase 1: NULL)")

```



```

172     media_ref: str | None = Field(default=None, description="Provider media reference")
173     document_id: str | None = Field(
174         default=None,
175         description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
176     )
177     duration_s: int | None = None
178     width: int | None = None
179     height: int | None = None
180     size_bytes: int | None = None
181     mime_type: str | None = None
182     dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
per account")
183     matched: bool = False
184     seen_at: datetime = Field(default_factory=_utcnow)
185
186
187     class Policy(BaseModel):
188         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
189
190         Generalizes the legacy per-account ``filter_config`` row so one account can
191         own multiple policies (enables ADR 0009's per-route filter overrides) and so
192         future enrichment/security stages grow into the same ``config_json`` column.
193         Exactly one policy per account carries ``is_default=True``; the legacy
194         ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
195         """
196
197         model_config = ConfigDict(frozen=True)
198
199         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
200         account_id: int = Field(description="FK ? accounts.id")
201         name: str = Field(description="Policy name; 'default' for the account default
policy")
202         config_json: str = Field(description="FilterConfig JSON blob plus ADR 0012 policy
keys")
203         is_default: bool = Field(
204             default=False, description="Exactly one per account (UNIQUE partial index)"
205         )
206         created_at: datetime = Field(default_factory=_utcnow)
207         updated_at: datetime = Field(default_factory=_utcnow)
208
209
210     # --- Track C: destinations, routing, delivery state (ADR 0011) -----
211     #
212     # These tables were created empty by Track B (_SCHEMA); Track C populates
213     # destinations (backfilled from filter_config_deprecated.config_json), writes
214     # delivery_attempts (replacing the legacy videos.forwarded boolean), and
215     # defines the models for per-destination delivery state. The source/destination
216     # invariant (a channel is either source or destination, never both) is enforced
217     # at the app level by future selective-scanning logic (Track E, #30).
218
219
220     class Destination(BaseModel):
221         """A delivery target ? where matched content is sent (ADR 0011, Track C).
222
223         Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
224         can have multiple named output channels. Backfilled from the existing
225         per-account ``delivery_target``; the default destination is the target used
226         when no explicit routing rule matches.
227         """
228
229         model_config = ConfigDict(frozen=True)
230
231         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
232         account_id: int = Field(description="FK ? accounts.id")
233         provider: str = Field(default="telegram", description="Provider namespace")

```

```

234     target: str = Field(description="Telethon entity: 'me', '@username', or chat id")
235     kind: str = Field(default="saved", description="'saved' | 'channel' | 'chat'")
236     title: str | None = None
237     is_output: bool = Field(
238         default=False,
239         description="ADR 0009: distinguishes output channels from ingest sources",
240     )
241     created_at: datetime = Field(default_factory=_utcnow)
242
243
244 class RoutingRule(BaseModel):
245     """A rule that maps a source through a policy to a destination (ADR 0009).
246
247     Canonical name: ``routing_rules`` (locked in umbrella issue #25). When
248     ``policy_id`` is NULL, the account's default policy is used. When multiple
249     rules match, the highest ``priority`` wins.
250     """
251
252     model_config = ConfigDict(frozen=True)
253
254     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
255     account_id: int = Field(description="FK ? accounts.id")
256     source_id: int = Field(description="FK ? sources.id")
257     policy_id: int | None = Field(
258         default=None,
259         description="FK ? policies.id; NULL means use account default policy",
260     )
261     destination_id: int = Field(description="FK ? destinations.id")
262     enabled: bool = Field(default=True)
263     priority: int = Field(default=0, description="Higher wins when multiple rules
match")
264     created_at: datetime = Field(default_factory=_utcnow)
265
266
267 class DeliveryStatus(str, Enum):
268     """Per-destination delivery state, replacing the legacy binary ``forwarded``."""
269
270     PENDING = "pending"
271     DELIVERED = "delivered"
272     FAILED = "failed"
273     RETRYING = "retrying"
274
275
276 class DeliveryAttempt(BaseModel):
277     """Per-destination delivery record ? idempotent via UNIQUE index.
278
279     Replaces the legacy ``videos.forwarded`` boolean (ADR 0011). One row per
280     ``(content_item_id, destination_id)`` pair. On retry, ``attempt_count`` is
281     incremented and ``status``/``last_error`` are updated.
282     """
283
284     model_config = ConfigDict(frozen=True)
285
286     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
287     content_item_id: int = Field(description="FK ? content_items.id")
288     destination_id: int = Field(description="FK ? destinations.id")
289     route_id: int | None = Field(default=None, description="FK ? routing_rules.id
(nullable)")
290     status: DeliveryStatus = Field(default=DeliveryStatus.PENDING)
291     attempt_count: int = Field(default=0, description="Incremented on each retry")
292     last_error: str | None = None
293     created_at: datetime = Field(default_factory=_utcnow)
294     delivered_at: datetime | None = None
295     next_retry_at: datetime | None = None
296

```

```

297
298 class PolicyDecision(BaseModel):
299     """Persisted ADR 0012 policy evaluation trace for one content item."""
300
301     model_config = ConfigDict(frozen=True)
302
303     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
304     content_item_id: int = Field(description="FK -> content_items.id")
305     policy_id: int = Field(description="FK -> policies.id")
306     passed: bool
307     trace: list[dict[str, object]] = Field(
308         default_factory=list,
309         description="Ordered StageResult trace serialized in
policy_decisions.trace_json",
310     )
311     evaluated_at: datetime = Field(default_factory=_utcnow)
312
313
314 class FilterConfig(BaseModel):
315     """Flat per-account filter configuration.
316
317     Stored inside the account's default ``policies.config_json`` object through
318     compatibility shims. Legacy ``filter_config`` callers still work, but new
319     policy keys can live alongside these flat fields without being erased by
320     ``/set`` command updates.
321     """
322
323     # Duration bounds in seconds. None means unbounded on that side.
324     min_duration_s: int | None = Field(default=5, ge=0)
325     max_duration_s: int | None = Field(default=7200, ge=0)
326
327     # Resolution: a video passes if its height >= target_height.
328     # None disables the resolution rule.
329     target_height: int | None = Field(default=1080, ge=1)
330
331     # Max file size in mebibytes. None disables the size rule.
332     max_size_mb: int | None = Field(default=2048, ge=1)
333
334     # Content-type exclusions. GIFs and round videos are excluded by default
335     # (their duration/dimensions are usually incidental to real video filters);
336     # silent videos are off by default since some legit clips have no audio.
337     # Existing users' stored configs are unaffected ? the DB JSON blob overrides
338     # these defaults, so this only changes the behaviour for fresh installs.
339     exclude_gif: bool = Field(
340         default=True,
341         description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
342     )
343     exclude_round: bool = Field(
344         default=True,
345         description="If True, exclude circular video messages (round_message=True)",
346     )
347     exclude_nosound: bool = Field(
348         default=False,
349         description="If True, exclude videos without an audio track (nosound=True)",
350     )
351
352     # How matched videos are delivered: forward the video, send a link, or both.
353     # Stored in the same JSON blob, so adding this field is backwards-compatible
354     # (existing rows without it default to FORWARD).
355     delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)
356
357     # Where matched videos / link messages are sent. Accepts:
358     # - "me" (default): Saved Messages ? exact v1.1 behaviour.
359     # - "@username": a public channel the account is a member of.
360     # - "-100..." or a bare int-as-string: a numeric Telegram chat id

```

```

361 # (private channels without a username).
362 # The string is passed verbatim to Telethon's forward_messages / send_message.
363 # See ADR 0005 for rationale and access requirements.
364 delivery_target: str = Field(
365     default="me",
366     min_length=1,
367     description="Telethon entity to deliver to: 'me', '@username', or a chat id",
368 )
369
370 @field_validator("delivery_target", mode="before")
371 @classmethod
372 def _coerce_delivery_target(cls, v: object) -> str:
373     """Accept int chat ids (e.g. -1001234567890) and coerce to str.
374
375     Telethon accepts both str and int as an entity, but storing as str keeps
376     the JSON blob uniform and lets '@username' / 'me' / numeric ids share
377     one field type.
378     """
379     if v is None:
380         return "me"
381     if isinstance(v, int):
382         return str(v)
383     return str(v)
384
385 @model_validator(mode="after")
386 def _check_duration_bounds(self) -> FilterConfig:
387     """Ensure min_duration_s <= max_duration_s when both are set."""
388     if (
389         self.min_duration_s is not None
390         and self.max_duration_s is not None
391         and self.min_duration_s > self.max_duration_s
392     ):
393         raise ValueError(
394             f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s "
395             f"({self.max_duration_s})"
396         )
397     return self
398
399 class VideoRecord(BaseModel):
400     """A seen video message + its metadata. Drives dedup and future /search.
401
402     Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).
403     New code should use :class:`ContentItem`; the ``db`` layer maps
404     ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.
405     """
406
407     model_config = ConfigDict(frozen=True)
408
409     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
410     user_id: int
411     channel_id: int = Field(description="Origin Telegram channel id")
412     message_id: int = Field(description="Telegram message id within the channel")
413     text: str | None = Field(default=None, description="Caption / message text")
414     document_id: int | None = Field(
415         default=None,
416         description="Telegram document.id ? reused across forwards; Tier-1 content dedup",
417     )
418
419     # Metadata extracted from the Telethon message (may be None if absent).
420     duration_s: int | None = None
421     width: int | None = None
422     height: int | None = None
423     size_bytes: int | None = None

```

```

424         mime_type: str | None = None
425
426         matched: bool = False
427         forwarded: bool = False
428         seen_at: datetime = Field(default_factory=_utcnow)
429
430     @property
431     def dedup_key(self) -> tuple[int, int, int]:
432         """The natural key used for deduplication in the ``videos`` table."""
433         return (self.user_id, self.channel_id, self.message_id)
434
435     @property
436     def content_fingerprint(self) -> tuple[int, int, int, int] | None:
437         """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height,
width).
438
439         Returns None if any field is missing (cannot fingerprint).
440         """
441         if None in (self.size_bytes, self.duration_s, self.height, self.width):
442             return None
443         return (self.size_bytes, self.duration_s, self.height, self.width) # type:
ignore[return-value]
444
445
446     # Default filter config shared by the bootstrap onboarding step.
447     DEFAULT_FILTER_CONFIG = FilterConfig()
448

```

11. user (2026-07-07T03:35:34.724Z)

```

1     """Policy pipeline primitives for ADR 0012.
2
3     The policy layer is deliberately small and framework-free in F1: stages receive
4     an immutable context, return a structured result, and the evaluator stops on
5     the first failure. Later Track F phases add concrete stages.
6     """
7
8     from __future__ import annotations
9
10    import asyncio
11    import contextlib
12    import json
13    import logging
14    import re
15    from collections.abc import Awaitable
16    from dataclasses import dataclass, field
17    from ipaddress import ip_address
18    from typing import Protocol
19    from urllib.parse import urlsplit
20
21    import regex
22
23    from tg_video_filter.models import ContentItem, Policy, Source
24
25    log = logging.getLogger(__name__)
26
27    _MAX_CAPTION_SCAN_CHARS = 4096
28    _MAX_CAPTION_TERMS = 100
29    _MAX_CAPTION_PATTERNS = 25
30    _CAPTION_REGEX_TIMEOUT_SECONDS = 0.01
31    _URL_RE = re.compile(
32        r"""(?ix)
33        (?<![@\\w.-])
34        (
35            (?:(https?://|www\\.)(^\\s<>"')+

```

```

36         |
37         (?:[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?\.\.)+[a-z]{2,}
38         (?:/[^\s<>"]*)?
39     )
40     """
41 )
42 _TRAILING_URL_PUNCTUATION = ".,!?;:"
43
44
45 class PolicyBudgetExceeded(Exception):
46     """Raised when a deep-inspection budget is exhausted."""
47
48     def __init__(self, exceeded_budget: str, **metadata: object) -> None:
49         super().__init__(exceeded_budget)
50         self.exceeded_budget = exceeded_budget
51         self.metadata = metadata
52
53
54 @dataclass(frozen=True)
55 class PolicyBudget:
56     """Bounded execution budget for opt-in deep inspection."""
57
58     max_bytes_per_item: int
59     timeout_s: float
60     max_concurrent: int
61     _semaphore: asyncio.Semaphore | None = field(
62         default=None,
63         init=False,
64         repr=False,
65         compare=False,
66     )
67
68     def __post_init__(self) -> None:
69         if self.max_concurrent > 0:
70             object.__setattr__(self, "_semaphore",
71 asyncio.Semaphore(self.max_concurrent))
72
73     async def inspect(self, fetcher: ContentFetcher, content_item: ContentItem) ->
StageResult:
74         """Run ``fetcher`` under the timeout and concurrency budget."""
75         if self._semaphore is None:
76             raise ValueError("max_concurrent must be positive")
77
78         loop = asyncio.get_running_loop()
79         deadline = loop.time() + self.timeout_s
80         await self._acquire_before(deadline)
81         try:
82             remaining = deadline - loop.time()
83             if remaining <= 0:
84                 raise PolicyBudgetExceeded("timeout_s", timeout_s=self.timeout_s)
85             return await _await_with_budget_timeout(
86                 fetcher.inspect(content_item, self),
87                 timeout_s=remaining,
88                 configured_timeout_s=self.timeout_s,
89             )
90         finally:
91             self._semaphore.release()
92
93     async def _acquire_before(self, deadline: float) -> None:
94         assert self._semaphore is not None
95         remaining = deadline - asyncio.get_running_loop().time()
96         if remaining <= 0:
97             raise PolicyBudgetExceeded("timeout_s", timeout_s=self.timeout_s)
98         try:
99             await asyncio.wait_for(self._semaphore.acquire(), timeout=remaining)

```

```

99         except TimeoutError as exc:
100             raise PolicyBudgetExceeded("timeout_s", timeout_s=self.timeout_s) from exc
101
102
103     async def _await_with_budget_timeout(
104         awaitable: Awaitable[StageResult],
105         *,
106         timeout_s: float,
107         configured_timeout_s: float,
108     ) -> StageResult:
109         task = asyncio.ensure_future(awaitable)
110         done, _pending = await asyncio.wait({task}, timeout=timeout_s)
111         if task not in done:
112             task.cancel()
113             with contextlib.suppress(asyncio.CancelledError):
114                 await task
115             raise PolicyBudgetExceeded("timeout_s", timeout_s=configured_timeout_s)
116         return await task
117
118
119     class ContentFetcher(Protocol):
120         """Fetcher used only by opt-in Tier 4 policy stages."""
121
122         async def inspect(self, content_item: ContentItem, budget: PolicyBudget) ->
123         StageResult:
124             """Inspect content under ``budget`` and return a policy result."""
125             ...
126
127     @dataclass(frozen=True)
128     class PolicyContext:
129         """Immutable snapshot available to a policy stage."""
130
131         content_item: ContentItem
132         source: Source
133         policy: Policy
134         fetcher: ContentFetcher | None = None
135         budget: PolicyBudget | None = None
136
137
138     @dataclass(frozen=True)
139     class StageResult:
140         """Outcome of one policy stage."""
141
142         passed: bool
143         reason: str
144         stage_name: str
145         metadata: dict[str, object] = field(default_factory=dict)
146
147         @staticmethod
148         def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
149             return StageResult(
150                 passed=True,
151                 reason=reason,
152                 stage_name=stage_name,
153                 metadata=metadata,
154             )
155
156         @staticmethod
157         def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
158             return StageResult(
159                 passed=False,
160                 reason=reason,
161                 stage_name=stage_name,
162                 metadata=metadata,

```

```

163         )
164
165
166     class PolicyStage(Protocol):
167         """A single stateless gate in the policy pipeline."""
168
169         async def evaluate(self, ctx: PolicyContext) -> StageResult:
170             """Evaluate ``ctx`` and return a stage result."""
171             ...
172
173
174     @dataclass(frozen=True)
175     class TopicPreferences:
176         """Parsed ADR 0012 topic preference config."""
177
178         include: tuple[str, ...] = ()
179         exclude: tuple[str, ...] = ()
180         mode: str = "any"
181
182         @property
183         def enabled(self) -> bool:
184             return bool(self.include or self.exclude)
185
186
187     @dataclass(frozen=True)
188     class CaptionSafetyConfig:
189         """Parsed ADR 0012 caption safety config."""
190
191         unsafe_terms: tuple[str, ...] = ()
192         unsafe_patterns: tuple[str, ...] = ()
193         invalid_patterns: tuple[str, ...] = ()
194         ignored_term_count: int = 0
195         ignored_pattern_count: int = 0
196
197         @property
198         def enabled(self) -> bool:
199             return bool(self.unsafe_terms or self.unsafe_patterns)
200
201         @property
202         def has_config_warnings(self) -> bool:
203             return bool(self.invalid_patterns or self.ignored_term_count or
204 self.ignored_pattern_count)
205
206     @dataclass(frozen=True)
207     class DomainReputationConfig:
208         """Parsed ADR 0012 URL/domain reputation config."""
209
210         blocked_domains: tuple[str, ...] = ()
211         invalid_domains: tuple[str, ...] = ()
212
213         @property
214         def enabled(self) -> bool:
215             return bool(self.blocked_domains)
216
217
218     @dataclass(frozen=True)
219     class ExtractedUrl:
220         """URL discovered from captured text with its normalized host domain."""
221
222         original_url: str
223         domain: str
224
225
226     def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:

```



```

227     """Parse ``topics`` from a raw policy config object."""
228     raw_topics = config.get("topics")
229     if raw_topics is None:
230         return TopicPreferences()
231     if not isinstance(raw_topics, dict):
232         raise ValueError("topics must be a JSON object")
233     include = _string_tuple(raw_topics.get("include"), field_name="topics.include")
234     exclude = _string_tuple(raw_topics.get("exclude"), field_name="topics.exclude")
235     if not include and not exclude:
236         return TopicPreferences()
237     mode = raw_topics.get("mode", "any")
238     if mode not in ("any", "all"):
239         raise ValueError("topics.mode must be 'any' or 'all'")
240     return TopicPreferences(
241         include=include,
242         exclude=exclude,
243         mode=str(mode),
244     )
245
246
247 def parse_caption_safety_config(config: dict[str, object]) -> CaptionSafetyConfig:
248     """Parse caption safety settings from ``security_checks``.
249
250     Expected shape:
251     ``{"security_checks":{"unsafe_caption_terms":[],"unsafe_caption_patterns":[]}}``.
252     """
253     raw_security = config.get("security_checks")
254     if raw_security is None:
255         return CaptionSafetyConfig()
256     if not isinstance(raw_security, dict):
257         raise ValueError("security_checks must be a JSON object")
258
259     raw_terms = _string_tuple(
260         raw_security.get("unsafe_caption_terms"),
261         field_name="security_checks.unsafe_caption_terms",
262     )
263     raw_patterns = _string_tuple(
264         raw_security.get("unsafe_caption_patterns"),
265         field_name="security_checks.unsafe_caption_patterns",
266     )
267
268     unsafe_terms = raw_terms[:_MAX_CAPTION_TERMS]
269     unsafe_patterns: list[str] = []
270     invalid_patterns: list[str] = []
271     for pattern in raw_patterns[:_MAX_CAPTION_PATTERNS]:
272         try:
273             regex.compile(pattern, regex.IGNORECASE)
274         except regex.error:
275             invalid_patterns.append(pattern)
276         else:
277             unsafe_patterns.append(pattern)
278
279     return CaptionSafetyConfig(
280         unsafe_terms=unsafe_terms,
281         unsafe_patterns=tuple(unsafe_patterns),
282         invalid_patterns=tuple(invalid_patterns),
283         ignored_term_count=max(0, len(raw_terms) - _MAX_CAPTION_TERMS),
284         ignored_pattern_count=max(0, len(raw_patterns) - _MAX_CAPTION_PATTERNS),
285     )
286
287
288 def parse_domain_reputation_config(config: dict[str, object]) -> DomainReputationConfig:
289     """Parse URL/domain reputation settings from ``security_checks``."""
290     raw_security = config.get("security_checks")
291     if raw_security is None:

```

```

292         return DomainReputationConfig()
293     if not isinstance(raw_security, dict):
294         raise ValueError("security_checks must be a JSON object")
295
296     raw_domains = _string_tuple(
297         raw_security.get("blocked_domains"),
298         field_name="security_checks.blocked_domains",
299     )
300     blocked_domains: list[str] = []
301     invalid_domains: list[str] = []
302     seen: set[str] = set()
303     for raw_domain in raw_domains:
304         domain = normalize_domain(raw_domain)
305         if domain is None:
306             invalid_domains.append(raw_domain)
307             continue
308         if domain not in seen:
309             blocked_domains.append(domain)
310             seen.add(domain)
311     return DomainReputationConfig(
312         blocked_domains=tuple(blocked_domains),
313         invalid_domains=tuple(invalid_domains),
314     )
315
316
317 def load_topic_preferences(policy: Policy) -> TopicPreferences:
318     """Load topic preferences from ``policy.config_json``."""
319     decoded = json.loads(policy.config_json)
320     if not isinstance(decoded, dict):
321         raise ValueError("policy config_json must decode to a JSON object")
322     return parse_topic_preferences(decoded)
323
324
325 def load_caption_safety_config(policy: Policy) -> CaptionSafetyConfig:
326     """Load caption safety settings from ``policy.config_json``."""
327     decoded = json.loads(policy.config_json)
328     if not isinstance(decoded, dict):
329         raise ValueError("policy config_json must decode to a JSON object")
330     return parse_caption_safety_config(decoded)
331
332
333 def load_domain_reputation_config(policy: Policy) -> DomainReputationConfig:
334     """Load URL/domain reputation settings from ``policy.config_json``."""
335     decoded = json.loads(policy.config_json)
336     if not isinstance(decoded, dict):
337         raise ValueError("policy config_json must decode to a JSON object")
338     return parse_domain_reputation_config(decoded)
339
340
341 def load_deep_inspection_enabled(policy: Policy) -> bool:
342     """Return whether Tier 4 deep inspection is explicitly enabled."""
343     decoded = json.loads(policy.config_json)
344     if not isinstance(decoded, dict):
345         raise ValueError("policy config_json must decode to a JSON object")
346     raw_deep_inspect = decoded.get("deep_inspect", False)
347     if isinstance(raw_deep_inspect, bool):
348         return raw_deep_inspect
349     if isinstance(raw_deep_inspect, dict):
350         enabled = raw_deep_inspect.get("enabled", False)
351         if isinstance(enabled, bool):
352             return enabled
353         raise ValueError("deep_inspect must be a boolean or an object with boolean enabled")
354
355
356 def _string_tuple(value: object, *, field_name: str) -> tuple[str, ...]:

```

```

357     if value is None:
358         return ()
359     if not isinstance(value, list):
360         raise ValueError(f"{field_name} must be an array")
361     values: list[str] = []
362     for item in value:
363         if not isinstance(item, str):
364             raise ValueError(f"{field_name} must contain strings")
365         text = item.strip()
366         if text:
367             values.append(text)
368     return tuple(values)
369
370
371 class TopicPreferenceStage:
372     """ADR 0012 Tier 2 topic matching over source title and captured caption."""
373
374     stage_name = "topic-preferences"
375
376     async def evaluate(self, ctx: PolicyContext) -> StageResult:
377         try:
378             prefs = load_topic_preferences(ctx.policy)
379         except (json.JSONDecodeError, ValueError) as exc:
380             log.warning(
381                 "topic preferences disabled due to invalid config policy_id=%s",
382                 ctx.policy.id,
383                 exc_info=True,
384             )
385             return StageResult.pass_(
386                 self.stage_name,
387                 f"invalid topic config ignored: {exc}",
388                 invalid_topic_config=True,
389             )
390         if not prefs.enabled:
391             return StageResult.pass_(self.stage_name, "topic preferences disabled")
392
393         haystack = _topic_haystack(ctx)
394         matched_excludes = _matched_topics(prefs.exclude, haystack)
395         if matched_excludes:
396             return StageResult.fail(
397                 self.stage_name,
398                 f"excluded topic matched: {matched_excludes[0]}",
399                 matched_exclude_topics=list(matched_excludes),
400             )
401
402         if not prefs.include:
403             return StageResult.pass_(self.stage_name, "no include topics configured")
404
405         matched_includes = _matched_topics(prefs.include, haystack)
406         if prefs.mode == "any":
407             if matched_includes:
408                 return StageResult.pass_(
409                     self.stage_name,
410                     f"matched include topic: {matched_includes[0]}",
411                     matched_include_topics=list(matched_includes),
412                 )
413             return StageResult.fail(self.stage_name, "no include topics matched")
414
415         missing = tuple(topic for topic in prefs.include if topic not in
matched_includes)
416         if missing:
417             return StageResult.fail(
418                 self.stage_name,
419                 f"missing required topic: {missing[0]}",
420                 matched_include_topics=list(matched_includes),

```

```

421         missing_include_topics=list(missing),
422     )
423     return StageResult.pass_(
424         self.stage_name,
425         "matched all include topics",
426         matched_include_topics=list(matched_includes),
427     )
428
429
430 def _topic_haystack(ctx: PolicyContext) -> str:
431     parts = [ctx.source.title or "", ctx.content_item.text or ""]
432     return "\n".join(parts).casefold()
433
434
435 def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
436     return tuple(topic for topic in topics if topic.casefold() in haystack)
437
438
439 def normalize_domain(value: str) -> str | None:
440     """Normalize a configured or extracted domain for blocklist matching."""
441     text = value.strip()
442     if not text:
443         return None
444
445     try:
446         parsed = urlsplit(text if "://" in text else f"//{text}")
447     except ValueError:
448         return None
449     host = parsed.hostname
450     if host is None:
451         return None
452
453     host = host.strip().strip("[]").rstrip(".").casefold()
454     if not host:
455         return None
456
457     try:
458         return ip_address(host).compressed.casefold()
459     except ValueError:
460         pass
461
462     try:
463         domain = host.encode("idna").decode("ascii").casefold()
464     except UnicodeError:
465         return None
466     if domain.startswith("www."):
467         domain = domain[4:]
468     if not _valid_domain_name(domain):
469         return None
470     return domain
471
472
473 def extract_caption_urls(text: str | None) -> tuple[ExtractedUrl, ...]:
474     """Extract plain URLs from captured caption text without network I/O."""
475     if not text:
476         return ()
477
478     urls: list[ExtractedUrl] = []
479     for match in _URL_RE.finditer(text):
480         raw_url = match.group(1).rstrip(_TRAILING_URL_PUNCTUATION)
481         while raw_url.endswith("(") and raw_url.count("(") < raw_url.count(")"):
482             raw_url = raw_url[:-1]
483         domain = normalize_domain(raw_url)
484         if domain is not None:
485             urls.append(ExtractedUrl(original_url=raw_url, domain=domain))

```

```

486         return tuple(urls)
487
488
489     def _valid_domain_name(domain: str) -> bool:
490         if len(domain) > 253 or "." not in domain:
491             return False
492         labels = domain.split(".")
493         return all(
494             re.fullmatch(r"[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?", label) is not None for
label in labels
495         )
496
497
498     def _domain_matches_blocked(domain: str, blocked_domain: str) -> bool:
499         return domain == blocked_domain or domain.endswith(f".{blocked_domain}")
500
501
502     class UrlReputationStage:
503         """ADR 0012 Tier 3 captured-URL domain blocklist checks."""
504
505         stage_name = "url-reputation"
506
507         async def evaluate(self, ctx: PolicyContext) -> StageResult:
508             try:
509                 config = load_domain_reputation_config(ctx.policy)
510             except (json.JSONDecodeError, ValueError) as exc:
511                 log.warning(
512                     "domain reputation disabled due to invalid config policy_id=%s",
513                     ctx.policy.id,
514                     exc_info=True,
515                 )
516                 return StageResult.pass_(
517                     self.stage_name,
518                     f"invalid domain reputation config ignored: {exc}",
519                     invalid_domain_reputation_config=True,
520                 )
521
522             if config.invalid_domains:
523                 log.warning(
524                     "domain reputation ignored invalid domains policy_id=%s
invalid_domains=%s",
525                     ctx.policy.id,
526                     list(config.invalid_domains),
527                 )
528             if not config.enabled:
529                 metadata = _url_reputation_metadata((), config)
530                 if config.invalid_domains:
531                     return StageResult.pass_(
532                         self.stage_name,
533                         "domain reputation has no valid blocked domains",
534                         **metadata,
535                     )
536                 return StageResult.pass_(self.stage_name, "domain reputation disabled")
537
538             urls = extract_captions_urls(ctx.content_item.link_url) + extract_captions_urls(
539                 ctx.content_item.text
540             )
541             if not urls:
542                 return StageResult.pass_(
543                     self.stage_name,
544                     "no links to scan",
545                     **_url_reputation_metadata(urls, config),
546                 )
547
548             for url in urls:

```

```

549         for blocked_domain in config.blocked_domains:
550             if _domain_matches_blocked(url.domain, blocked_domain):
551                 return StageResult.fail(
552                     self.stage_name,
553                     f"blocked domain matched: {blocked_domain}",
554                     **_url_reputation_metadata(urls, config),
555                     blocked_domain=url.domain,
556                     matched_blocklist_domain=blocked_domain,
557                     matched_url=url.original_url,
558                 )
559
560         return StageResult.pass_(
561             self.stage_name,
562             "links passed domain reputation checks",
563             **_url_reputation_metadata(urls, config),
564         )
565
566
567     def _url_reputation_metadata(
568         urls: tuple[ExtractedUrl, ...],
569         config: DomainReputationConfig,
570     ) -> dict[str, object]:
571         domains = sorted({url.domain for url in urls})
572         metadata: dict[str, object] = {
573             "link_count": len(urls),
574             "domains": domains,
575             "blocked_domain_count": len(config.blocked_domains),
576         }
577         if config.invalid_domains:
578             metadata["ignored_invalid_domains"] = list(config.invalid_domains)
579         return metadata
580
581
582     class CaptionSafetyStage:
583         """ADR 0012 Tier 3 captured-caption term and pattern scanning."""
584
585         stage_name = "caption-safety"
586
587         async def evaluate(self, ctx: PolicyContext) -> StageResult:
588             try:
589                 config = load_caption_safety_config(ctx.policy)
590             except (json.JSONDecodeError, ValueError) as exc:
591                 log.warning(
592                     "caption safety disabled due to invalid config policy_id=%s",
593                     ctx.policy.id,
594                     exc_info=True,
595                 )
596                 return StageResult.pass_(
597                     self.stage_name,
598                     f"invalid caption safety config ignored: {exc}",
599                     invalid_caption_safety_config=True,
600                 )
601             if not config.enabled:
602                 metadata = _caption_safety_metadata(config, caption_present=False)
603                 if config.has_config_warnings:
604                     log.warning(
605                         "caption safety has no valid active checks policy_id=%s",
606                         ctx.policy.id,
607                         metadata,
608                     )
609                 return StageResult.pass_(
610                     self.stage_name,
611                     "caption safety has no valid active checks",
612                     **metadata,

```

```

613         )
614         return StageResult.pass_(self.stage_name, "caption safety disabled")
615
616     if config.has_config_warnings:
617         log.warning(
618             "caption safety ignored invalid or excess config policy_id=%s "
619             "invalid_patterns=%s ignored_terms=%s ignored_patterns=%s",
620             ctx.policy.id,
621             list(config.invalid_patterns),
622             config.ignored_term_count,
623             config.ignored_pattern_count,
624         )
625
626     caption = (ctx.content_item.text or "").strip()
627     if not caption:
628         return StageResult.pass_(
629             self.stage_name,
630             "no caption to scan",
631             **_caption_safety_metadata(config, caption_present=False),
632         )
633
634     scan_caption = caption[:_MAX_CAPTION_SCAN_CHARS]
635     caption_truncated = len(caption) > _MAX_CAPTION_SCAN_CHARS
636     haystack = scan_caption.casefold()
637     for term in config.unsafe_terms:
638         if term.casefold() in haystack:
639             return StageResult.fail(
640                 self.stage_name,
641                 f"unsafe caption term matched: {term}",
642                 **_caption_safety_metadata(
643                     config,
644                     caption_present=True,
645                     caption_truncated=caption_truncated,
646                 ),
647                 match_type="term",
648                 matched_caption_term=term,
649             )
650
651     timed_out_patterns: list[str] = []
652     for pattern in config.unsafe_patterns:
653         try:
654             matched_pattern = regex.search(
655                 pattern,
656                 scan_caption,
657                 flags=regex.IGNORECASE,
658                 timeout=_CAPTION_REGEX_TIMEOUT_SECONDS,
659             )
660         except TimeoutError:
661             timed_out_patterns.append(pattern)
662             log.warning(
663                 "caption safety regex timed out policy_id=%s pattern=%r",
664                 ctx.policy.id,
665                 pattern,
666             )
667             continue
668
669     if matched_pattern:
670         return StageResult.fail(
671             self.stage_name,
672             "unsafe caption pattern matched",
673             **_caption_safety_metadata(
674                 config,
675                 caption_present=True,
676                 caption_truncated=caption_truncated,
677                 timed_out_patterns=tuple(timed_out_patterns),

```

```

678         ),
679         match_type="pattern",
680         matched_caption_pattern=pattern,
681     )
682
683     return StageResult.pass_(
684         self.stage_name,
685         "caption passed safety checks",
686         **_caption_safety_metadata(
687             config,
688             caption_present=True,
689             caption_truncated=caption_truncated,
690             timed_out_patterns=tuple(timed_out_patterns),
691         ),
692     )
693
694
695 def _caption_safety_metadata(
696     config: CaptionSafetyConfig,
697     *,
698     caption_present: bool,
699     caption_truncated: bool = False,
700     timed_out_patterns: tuple[str, ...] = (),
701 ) -> dict[str, object]:
702     metadata: dict[str, object] = {
703         "caption_present": caption_present,
704         "checked_terms": len(config.unsafe_terms),
705         "checked_patterns": len(config.unsafe_patterns),
706     }
707     if caption_truncated:
708         metadata["caption_truncated"] = True
709         metadata["max_caption_scan_chars"] = _MAX_CAPTION_SCAN_CHARS
710     if config.invalid_patterns:
711         metadata["ignored_invalid_caption_patterns"] = list(config.invalid_patterns)
712     if config.ignored_term_count:
713         metadata["ignored_caption_term_count"] = config.ignored_term_count
714     if config.ignored_pattern_count:
715         metadata["ignored_caption_pattern_count"] = config.ignored_pattern_count
716     if timed_out_patterns:
717         metadata["timed_out_caption_patterns"] = list(timed_out_patterns)
718     return metadata
719
720
721 class DeepInspectionStage:
722     """ADR 0012 Tier 4 opt-in boundary for budgeted heavy inspection."""
723
724     stage_name = "deep-inspection"
725
726     async def evaluate(self, ctx: PolicyContext) -> StageResult:
727         try:
728             enabled = load_deep_inspection_enabled(ctx.policy)
729         except (json.JSONDecodeError, ValueError) as exc:
730             log.warning(
731                 "deep inspection disabled due to invalid config policy_id=%s",
732                 ctx.policy.id,
733                 exc_info=True,
734             )
735             return StageResult.pass_(
736                 self.stage_name,
737                 f"invalid deep inspection config ignored: {exc}",
738                 invalid_deep_inspection_config=True,
739             )
740         if not enabled:
741             return StageResult.pass_(self.stage_name, "deep inspection disabled")
742

```



```

743         budget_result = _deep_inspection_budget_result(ctx)
744     if budget_result is not None:
745         return budget_result
746
747     if ctx.fetcher is None:
748         return StageResult.fail(
749             self.stage_name,
750             "deep inspection fetcher missing",
751             missing_fetcher=True,
752         )
753
754     assert ctx.budget is not None
755     try:
756         return await ctx.budget.inspect(ctx.fetcher, ctx.content_item)
757     except PolicyBudgetExceeded as exc:
758         return StageResult.fail(
759             self.stage_name,
760             "deep inspection budget exceeded",
761             budget_exceeded=True,
762             exceeded_budget=exc.exceeded_budget,
763             **exc.metadata,
764         )
765     except Exception as exc:
766         log.warning(
767             "deep inspection fetcher failed policy_id=%s",
768             ctx.policy.id,
769             exc_info=True,
770         )
771         return StageResult.fail(
772             self.stage_name,
773             "deep inspection fetcher failed",
774             fetcher_error_type=type(exc).__name__,
775             fetcher_error=str(exc),
776         )
777
778
779 def _deep_inspection_budget_result(ctx: PolicyContext) -> StageResult | None:
780     budget = ctx.budget
781     if budget is None:
782         return StageResult.fail(
783             DeepInspectionStage.stage_name,
784             "deep inspection budget missing",
785             missing_budget=True,
786         )
787     if budget.max_bytes_per_item <= 0 or budget.timeout_s <= 0 or budget.max_concurrent
788     <= 0:
789         return StageResult.fail(
790             DeepInspectionStage.stage_name,
791             "deep inspection budget invalid",
792             invalid_budget=True,
793             max_bytes_per_item=budget.max_bytes_per_item,
794             timeout_s=budget.timeout_s,
795             max_concurrent=budget.max_concurrent,
796         )
797     size_bytes = ctx.content_item.size_bytes
798     if size_bytes is None:
799         return StageResult.fail(
800             DeepInspectionStage.stage_name,
801             "deep inspection budget cannot be enforced without content size",
802             missing_size_bytes=True,
803             max_bytes_per_item=budget.max_bytes_per_item,
804         )
805     if size_bytes <= 0:
806         return StageResult.fail(
807             DeepInspectionStage.stage_name,

```

```

807         "deep inspection budget cannot be enforced with non-positive content size",
808         invalid_size_bytes=size_bytes,
809         max_bytes_per_item=budget.max_bytes_per_item,
810     )
811 if size_bytes > budget.max_bytes_per_item:
812     return StageResult.fail(
813         DeepInspectionStage.stage_name,
814         "deep inspection budget exceeded",
815         budget_exceeded=True,
816         exceeded_budget="max_bytes_per_item",
817         size_bytes=size_bytes,
818         max_bytes_per_item=budget.max_bytes_per_item,
819     )
820 return None
821
822
823 async def evaluate_pipeline(
824     ctx: PolicyContext,
825     stages: list[PolicyStage],
826 ) -> tuple[bool, list[StageResult]]:
827     """Evaluate stages in order and short-circuit on the first failure."""
828     results: list[StageResult] = []
829     for stage in stages:
830         result = await stage.evaluate(ctx)
831         results.append(result)
832         if not result.passed:
833             return False, results
834     return True, results
835
836
837 def stage_results_to_trace(results: list[StageResult]) -> list[dict[str, object]]:
838     """Convert in-memory stage results to a JSON-serializable trace."""
839     return [
840         {
841             "stage_name": result.stage_name,
842             "passed": result.passed,
843             "reason": result.reason,
844             "metadata": result.metadata,
845         }
846         for result in results
847     ]
848
849
850 __all__ = [
851     "CaptionSafetyConfig",
852     "CaptionSafetyStage",
853     "ContentFetcher",
854     "DeepInspectionStage",
855     "DomainReputationConfig",
856     "ExtractedUrl",
857     "PolicyContext",
858     "PolicyBudget",
859     "PolicyBudgetExceeded",
860     "PolicyStage",
861     "StageResult",
862     "TopicPreferenceStage",
863     "TopicPreferences",
864     "UrlReputationStage",
865     "evaluate_pipeline",
866     "extract_caption_urls",
867     "load_caption_safety_config",
868     "load_deep_inspection_enabled",
869     "load_domain_reputation_config",
870     "load_topic_preferences",
871     "normalize_domain",

```

```

872         "parse_caption_safety_config",
873         "parse_domain_reputation_config",
874         "parse_topic_preferences",
875         "stage_results_to_trace",
876     ]
877

```

12. assistant (2026-07-07T03:35:38.189Z)

The policy.py confirms the stage order and the deep-inspection behavior. Now let me verify the wiring in manager/telethon_client (is fetcher/budget None in live path?), the commands, and how the pipeline is constructed. Let me also check the models.py diff and command_service diff.

13. user (2026-07-07T03:35:41.585Z)

```

diff --git a/Dockerfile b/Dockerfile
index 9f4ffa3..a0c882b 100644
--- a/Dockerfile
+++ b/Dockerfile
@@ -25,6 +25,5 @@ ENV DB_PATH=/data/app.db \
    SESSIONS_DIR=/data/sessions \
    LOG_LEVEL=INFO

-# Run the manager entrypoint (Phase 4). For now this will fail fast until
-# __main__.py lands; that's expected at the Phase 0 stage.
+# Run the manager entrypoint.
+  CMD ["python", "-m", "tg_video_filter"]
diff --git a/pyproject.toml b/pyproject.toml
index 14ea202..83c4aad 100644
--- a/pyproject.toml
+++ b/pyproject.toml
@@ -5,7 +5,7 @@ build-backend = "hatchling.build"
 [project]
 name = "tg-video-filter"
 version = "0.1.0"
-description = "Telethon userbot that filters Telegram channel videos by duration, resolution,
+description = "Telethon userbot that filters selected Telegram video sources, delivers matches,
and size, and forwards matches to Saved Messages."
+description = "Telethon userbot that filters selected Telegram video sources, delivers matches,
and explains policy decisions."
 readme = "README.md"
 requires-python = ">=3.11"
 license = { text = "TBD" }
diff --git a/src/tg_video_filter/command_service.py b/src/tg_video_filter/command_service.py
index 0ab172a..79fef65 100644
--- a/src/tg_video_filter/command_service.py
+++ b/src/tg_video_filter/command_service.py
@@ -150,11 +150,11 @@ def format_commands() -> str:
     "`/set min resolution <N>p` ? Set min video height (e.g. `/set min resolution
720p`)",
        "    or `/set min res <N>p` ? same as above",
        "",
-        "? **Content exclusions** (all off by default)",
+        "? **Content exclusions**",
        "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
        "`/set exclude round on|off` ? Exclude circular video messages",
        "`/set exclude nosound on|off` ? Exclude videos without audio\n"
-        "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs too.",
+        "    Defaults: GIF/round excluded, nosound allowed.",
        "",
        "? **Backfill**",
        "`/backfill [N][hwd]` ? Scan last N hours/days/weeks of channel history (default
90d)",
diff --git a/src/tg_video_filter/models.py b/src/tg_video_filter/models.py
index fd4c85e..5c36b88 100644
--- a/src/tg_video_filter/models.py

```

```

+++ b/src/tg_video_filter/models.py
@@ -149,9 +149,10 @@ class Source(BaseModel):
    class ContentItem(BaseModel):
        """A normalized content record discovered from any :class:`Source` (ADR 0011).

-       Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
-       (`content_type='video'`); `text` / `link_url` are landed now but stay
-       NULL until later tracks populate them (avoids a future `ALTER TABLE`).
+       Generalizes the legacy :class:`VideoRecord`. The Telegram path stores videos
+       today (`content_type='video'`) and captures caption text; `link_url` is
+       available for future link/article content while URL policy stages currently
+       extract links from captured text at evaluation time.
        `dedup_key` is the provider-neutral dedup key (Option 1A: tiered lookup
        wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
        """

@@ -198,9 +199,7 @@ class Policy(BaseModel):
    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    account_id: int = Field(description="FK ? accounts.id")
    name: str = Field(description="Policy name; 'default' for the account default policy")
-   config_json: str = Field(
-       description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
-   )
+   config_json: str = Field(description="FilterConfig JSON blob plus ADR 0012 policy keys")
    is_default: bool = Field(
        default=False, description="Exactly one per account (UNIQUE partial index)"
    )

@@ -313,11 +312,12 @@ class PolicyDecision(BaseModel):

    class FilterConfig(BaseModel):
-       """Per-user filter configuration.
+       """Flat per-account filter configuration.

-       v1 rules: duration bounds, minimum height (resolution), and max file size.
-       Stored in the DB as a JSON blob (`filter_config.config_json`) so adding a
-       new field later is backwards-compatible.
+       Stored inside the account's default `policies.config_json` object through
+       compatibility shims. Legacy `filter_config` callers still work, but new
+       policy keys can live alongside these flat fields without being erased by
+       `/set` command updates.
        """

        # Duration bounds in seconds. None means unbounded on that side.
diff --git a/tests/test_commands.py b/tests/test_commands.py
index 162c97b..7ff316d 100644
--- a/tests/test_commands.py
+++ b/tests/test_commands.py
@@ -1101,7 +1101,7 @@ def test_format_commands_includes_exclude() -> None:
    assert "/set exclude gif" in result
    assert "/set exclude round" in result
    assert "/set exclude nosound" in result
-   assert "off by default" in result
+   assert "GIF/round excluded, nosound allowed" in result

    # --- regex: /set min resolution -----

```

14. user (2026-07-07T03:35:42.367Z)

```

C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:371:class TopicPreferenceStage:
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:502:class UrlReputationStage:
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:582:class CaptionSafetyStage:
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:721:class DeepInspectionStage:
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:751:
missing_fetcher=True,

```

```

C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:762:
exceeded_budget=exc.exceeded_budget,
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:783:
DeepInspectionStage.stage_name,
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:785:
missing_budget=True,
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:789:
DeepInspectionStage.stage_name,
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:791:
invalid_budget=True,
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:799:
DeepInspectionStage.stage_name,
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:806:
DeepInspectionStage.stage_name,
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:813:
DeepInspectionStage.stage_name,
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:816:
exceeded_budget="max_bytes_per_item",
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:823:async def evaluate_pipeline(
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:852:     "CaptionSafetyStage",
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:854:     "DeepInspectionStage",
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:862:     "TopicPreferenceStage",
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:864:     "UrlReputationStage",
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\policy.py:865:     "evaluate_pipeline",
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\telethon_client.py:48:
CaptionSafetyStage,
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\telethon_client.py:49:
DeepInspectionStage,
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\telethon_client.py:52:
TopicPreferenceStage,
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\telethon_client.py:53:
UrlReputationStage,
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\telethon_client.py:54:
evaluate_pipeline,
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\telethon_client.py:452:     #
DeepInspectionStage is dormant unless ``deep_inspect`` is enabled. Any
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\telethon_client.py:454:     passed, results
= await evaluate_pipeline(
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\telethon_client.py:455:
PolicyContext(content_item=content_item, source=source, policy=policy),
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\telethon_client.py:457:
TopicPreferenceStage(),
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\telethon_client.py:458:
UrlReputationStage(),
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\telethon_client.py:459:
CaptionSafetyStage(),
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\telethon_client.py:460:
DeepInspectionStage(),

```

15. assistant (2026-07-07T03:35:46.577Z)

This confirms the pipeline construction: `PolicyContext(content\_item, source, policy)` with no fetcher/budget (both default None), and stages in order topic -> url -> caption -> deep. So the design.md order and "no fetcher wired" claims are accurate. Let me verify the telethon_client wiring context lines 440-470, and check the commands surface (`/show`, `/set`, etc.) and delivery link fallback.

16. user (2026-07-07T03:35:47.600Z)

```

440
441
442     async def _evaluate_policy_stages(
443         conn: aiomysqlite.Connection,
444         account_id: int,
445         source: Source,

```

```

446         content_item: ContentItem,
447     ) -> tuple[bool, list[StageResult], int | None]:
448         """Evaluate ADR 0012 policy stages for a candidate item."""
449         policy = await db.get_default_policy(conn, account_id)
450         if policy is None:
451             return True, [], None
452         # DeepInspectionStage is dormant unless ``deep_inspect`` is enabled. Any
453         # user-facing enable command must land with fetcher/budget wiring.
454         passed, results = await evaluate_pipeline(
455             PolicyContext(content_item=content_item, source=source, policy=policy),
456             [
457                 TopicPreferenceStage(),
458                 UrlReputationStage(),
459                 CaptionSafetyStage(),
460                 DeepInspectionStage(),
461             ],
462         )
463         return passed, results, policy.id
464
465
466     async def _record_policy_decision(
467         conn: aiomysqlite.Connection,
468         *,
469         inserted: VideoRecord,

```

17. user (2026-07-07T03:35:49.611Z)

```

4: same ``/set``, ``/show`` commands work identically through a Bot API
32: _SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmgT]?b?)$", re.IGNORECASE)
33: _LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
34: _MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
35: _MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$", re.IGNORECASE)
38: _WHY_RE = re.compile(r"^/why(?:\s+(.*))?$", re.IGNORECASE)
39: _EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$", re.IGNORECASE)
40: _EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
41: _MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
46: _WATCH_RE = re.compile(r"^/watch\s+(.+)$", re.IGNORECASE)
47: _UNWATCH_RE = re.compile(r"^/unwatch\s+(.+)$", re.IGNORECASE)
48: _CHANNELS_RE = re.compile(r"^/channels$", re.IGNORECASE)
49: _WATCHLIST_RE = re.compile(r"^/watchlist$", re.IGNORECASE)
142:     """/show config` ? Show current filter settings",
143:     """/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
145:     """/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
146:     "    or `/set max length <N><unit>` ? same as above",
148:     """/set min length <N><unit>` ? Set min video duration (e.g. `/set min length
5s`)",
150:     """/set min resolution <N>p` ? Set min video height (e.g. `/set min resolution
720p`)",
151:     "    or `/set min res <N>p` ? same as above",
154:     """/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
155:     """/set exclude round on|off` ? Exclude circular video messages",
156:     """/set exclude nosound on|off` ? Exclude videos without audio\n"
160:     """/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history (default
90d)",
161:     """/stop` ? Cancel running backfill",
164:     """/channels` ? List all known sources with watch status",
165:     """/watch <id|@name>` ? Start watching a source for new videos",
166:     """/unwatch <id|@name>` ? Stop watching a source",
167:     """/watchlist` ? Show only watched sources",
170:     """/why <content_item_id>` ? Explain why an item passed or failed",
173:     """/show commands` ? Show this help message",
329:     # --- /set size <value><unit> -----
339:         return CommandResult("? Invalid size format. Use e.g. `/set size 10MB`.")
348:     # --- /set length <value><unit> -----
355:         return CommandResult("? Invalid duration format. Use e.g. `/set length

```

```

15m`.")
368:         # --- /set min length <value><unit> -----
375:         return CommandResult("? Invalid duration format. Use e.g. `/set min length
5s`.")
388:         # --- /set min resolution <N>p -----
402:         "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)"
405:         # --- /set exclude <gif|round|nosound> <on|off> -----
426:         "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
427:         "Example: `/set exclude gif on`"
430:         # --- /show config -----
450:         # --- /why <content_item_id> -----
455:         # --- /show commands -----
467:         """"Parse `/watch <username|channel_id>` and toggle `watched=1`."""
471:         f"? Source `{target}` not found. Use `/channels` to list "
483:         """"Parse `/unwatch <username|channel_id>` and toggle `watched=0`."""
487:         f"? Source `{target}` not found. Use `/channels` to list sources."
500:         "channel. Run `/backfill` to discover historical channels, "
516:         "Use `/channels` to see available sources, then `/watch <id>` "
534:         return CommandResult("? Usage: `/why <content_item_id>`)
537:         return CommandResult("? Usage: `/why <content_item_id>`)

```

18. assistant (2026-07-07T03:35:53.613Z)

All commands documented in README/design match code. Now let me verify a few remaining specifics: the delivery `link` fallback claim (design says "Falls back to forward if no link text"), the `should\_ingest\_source` opt-in default, `policy\_decisions` table existence + retention, and the `/why` account-scoping + markdown escaping. Let me check delivery.py and db.py quickly.

19. user (2026-07-07T03:35:56.260Z)

```

104:     watched                INTEGER NOT NULL DEFAULT 0,
186:CREATE TABLE IF NOT EXISTS policy_decisions (
224:     # Once migration 003 has run (signalled by `channels_deprecated` existing),
227:     if await _table_exists(conn, "channels_deprecated"):
277: def compute_dedup_key(
375:     ``users_deprecated`` and kept for one release as a rollback safety net; a
381:     those FKs to point at ``users_deprecated``, diverging from fresh installs
443:     # the rename ? so a naive rename of users ? users_deprecated would leave
444:     # these tables referencing users_deprecated(id), diverging from fresh DBs
516:     await conn.execute("ALTER TABLE users RENAME TO users_deprecated")
528:     legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
537:     # 1a. Backfill sources from channels first (known title + watched=1). Order
538:     #     matters: channels-first so title/watched data wins; the videos-based
543:     #     (account_id, provider, provider_source_id, title, kind, watched,
554:     #     ? the channels-based row above survives with its title + watched=1.
558:     #     (account_id, provider, provider_source_id, kind, watched, seen_first_at)
658:     #     investigate; the data is preserved in *_deprecated tables.
669:     #     "(%d gap). Colliding items remain in videos_deprecated.",
677:     await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
678:     await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
679:     await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
687:     ``filter_config_deprecated.delivery_target`` value, then creates a default
689:     Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
700:     if not await _table_exists(conn, "filter_config_deprecated"):
703:     # 1. Backfill default destinations from filter_config_deprecated.
725:         FROM filter_config_deprecated
745:         FROM filter_config_deprecated fcd
760:         FROM videos_deprecated vd
779:async def _migration_005_policy_decisions(conn: aiosqlite.Connection) -> None:
783:     CREATE TABLE IF NOT EXISTS policy_decisions (
884:         watched=bool(row["watched"]),
928:         raise ValueError("policy_decisions.trace_json must decode to a JSON array")
1151:async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
1157:     ``watched`` is **user-controlled state** (toggled only via

```

```

1158: :func:`set_source_watched` / the ``/watch`` and ``/unwatch`` commands) and is
1161: so clobbering ``watched`` here would silently un-watch a channel the user
1162: opted into after its first ingested video (ADR 0009). ``watched`` is only set
1169: (account_id, provider, provider_source_id, title, kind, watched,
1181: int(source.watched),
1288:async def get_content_item_for_account(

```

[Showing results with pagination = limit: 40]

20. user (2026-07-07T03:35:57.819Z)

```

5:- ``FORWARD``: forward the video itself (re-uploads to Saved Messages).
68:def build_link_text(
195:async def deliver(
202: link_text: str | None = None,
208: ``link_text`` is required for LINK/BOTH modes (built by the caller via
209: :func:`build_link_text`). If it's None in LINK/BOTH mode, falls back to
210: FORWARD so we never silently drop a matched video.
218: if mode == DeliveryMode.LINK and link_text is not None:
220: client, link_text, conn, video_id, target, destination_id=destination_id
222: if mode == DeliveryMode.BOTH and link_text is not None:
244: link_text,
270: # FORWARD, or fallback when link_text is missing.
271: if mode != DeliveryMode.FORWARD:
273: "link_mode_fallback_to_forward video=%s mode=%s link_text_missing=%s",
276: link_text is None,
284: "build_link_text",

```

21. assistant (2026-07-07T03:36:07.927Z)

All confirmed. Delivery link fallback, watched default 0, upsert_source no longer clobbers watched, policy_decisions table exists, ``\*\_deprecated`` tables exist, ``get\_content\_item\_for\_account`` exists.

Now let me check the design.md ``should\_ingest\_source`` claim (design step: "The live handler returns immediately for unwatched sources"). Also verify the design.md claim about deep-inspection: "Falls back to forward" ? done. Let me verify ``should\_ingest\_source`` and the deep-inspect-drops-items behavior against the design's phrasing. The design says deep inspection is "budgeted but no production fetcher is wired by default" ? accurate (from ``PolicyContext(...)`` with fetcher=None ? DeepInspectionStage returns fail only when enabled). Let me verify the design/README's claim that enabling deep_inspect "drops items" ? README says "budgeted but no production fetcher is wired by default." That's accurate and doesn't overclaim.

Let me check the runbook and plan diffs, and the ADR 0012 diff for status claims.

22. user (2026-07-07T03:36:08.616Z)

```

diff --git a/docs/adr/0007-inline-config-commands.md b/docs/adr/0007-inline-config-commands.md
index 4a40af4..f9bbfbb 100644
--- a/docs/adr/0007-inline-config-commands.md
+++ b/docs/adr/0007-inline-config-commands.md
@@ -4,6 +4,15 @@
- **Date**: 2026-07-04
- **Decision owner**: project owner

+## Implementation status
+
+The original command set has been extended. Current Saved Messages commands
+include ``/show config``, ``/show commands``, the ``/set`` size/length/min-resolution
+and exclusion commands, source scanning commands (``/channels``, ``/watch``,
+``/unwatch``, ``/watchlist``), backfill commands, and ``/why <content_item_id>``.
+``CommandService`` is provider-independent and the Telethon module is only a thin
+adapter.
+

```


Context

Filter criteria (duration bounds, resolution threshold, max file size) were set

```
diff --git a/docs/adr/0009-selective-channel-scanning.md b/docs/adr/0009-selective-channel-scanning.md
index d03db20..5fe405c 100644
--- a/docs/adr/0009-selective-channel-scanning.md
+++ b/docs/adr/0009-selective-channel-scanning.md
@@ -4,6 +4,16 @@
- **Date**: 2026-07-04
- **Decision owner**: project owner
```

Implementation status

+ Selective live scanning is implemented on the provider-neutral `sources` table, not the legacy `channels` table. Unknown live sources are created with `watched=false` and skipped until the user runs `/watch`. The shipped `/channels` command lists discovered source rows; `/backfill` can also discover historical dialogs. The provider-neutral `routing\_rules` table exists, but the runtime still delivers through the account's current `delivery\_target` unless future routing logic is added.

Context

Today the userbot scans **every** channel/group the account is a member of.

```
diff --git a/docs/adr/0010-identity-and-session-separation.md b/docs/adr/0010-identity-and-session-separation.md
index 7791bc2..1c826cd 100644
--- a/docs/adr/0010-identity-and-session-separation.md
+++ b/docs/adr/0010-identity-and-session-separation.md
@@ -1,11 +1,18 @@
# ADR 0010: Separate product identity from Telegram sessions

-- **Status**: Proposed (awaiting decision owner pick between Option A and Option B)
+- **Status**: Accepted and implemented (Option A; Track A merged)
- **Date**: 2026-07-05
- **Decision owner**: project owner
- **Tracking issue**: #26 (Track A of the provider-neutral umbrella, #25)
- **Blocks**: ADR 0008 (Bot API onboarding), #27 (Track B domain schema), #28 (Track C delivery), #29 (Track D commands)
```

Implementation status

+ The schema now has `accounts` for product identity and `telegram\_sessions` for runnable MTPROTO sessions. Legacy `users` APIs remain compatibility shims over those tables, so older call sites and databases continue to work while new provider-neutral tables use `account\_id`.

Context

Today the `users` table conflates two orthogonal concepts:

```
diff --git a/docs/adr/0011-provider-neutral-domain-schema.md b/docs/adr/0011-provider-neutral-domain-schema.md
index aa37418..3be70a1 100644
--- a/docs/adr/0011-provider-neutral-domain-schema.md
+++ b/docs/adr/0011-provider-neutral-domain-schema.md
@@ -7,6 +7,16 @@
- **Depends on**: ADR 0010 (Track A ? `accounts` + `telegram_sessions`, merged in #33)
- **Blocks**: #28 (Track C ? delivery/routing), #29 (Track D ? commands), and indirectly #30 (Track E)
```

Implementation status

+ The provider-neutral schema is implemented. `sources`, `content\_items`, `policies`, `destinations`, `routing\_rules`, `delivery\_attempts`, and

```

+`policy_decisions` exist in the current SQLite schema. Legacy `channels`,
+`videos`, and `filter_config` APIs are compatibility shims over the newer
+tables. The Telegram path now captures caption text into `content_items.text`;
+`content_items.link_url` remains available for future link/article content,
+while URL policy stages currently extract links from captured text.
+
## Context

ADR 0010 split `users` into `accounts` (product identity) + `telegram_sessions`
@@ -374,9 +384,9 @@ shim), so existing call sites in `telethon_client.py`, `backfill.py`, and
single-purpose, but the surface area is larger.
- The dedup-key derivation logic is in code, not the schema ? needs its
own unit tests (a `compute_dedup_key(provider, **metadata)` pure function).
-- Phase 1 carries some dead-on-arrival columns (`content_items.link_url`,
- `text`) that aren't populated until later tracks. This is intentional
- (avoid future migrations) but means some columns are NULL until then.
+- `content_items.link_url` is still reserved for future link/article content.
+ The current Telegram video path captures captions in `content_items.text`
+ and URL policy stages extract links from that captured text.

## Alternatives considered

@@ -399,9 +409,9 @@ shim), so existing call sites in `telethon_client.py`, `backfill.py`, and
rejected), or 1C (provider-assigned only).
2. Question 2 ? `Policy` ? `filter_config`: 2A (generalize, recommended)
or 2B (wrap, rejected).
-3. Dead columns in Phase 1 ? acceptable to land `content_items.link_url`
- / `text` as NULL until later tracks populate them? Recommendation: yes
- (avoids a future `ALTER TABLE`).
+3. Dead columns in Phase 1 ? resolved by later tracks for
+ `content_items.text`; `link_url` remains reserved for future link/article
+ content.
4. Deprecated-table retention ? one release (default), matching ADR 0010?
Recommendation: yes.

diff --git a/docs/adr/0012-policy-pipeline.md b/docs/adr/0012-policy-pipeline.md
index d5d86d1..ae0b8b3 100644
--- a/docs/adr/0012-policy-pipeline.md
+++ b/docs/adr/0012-policy-pipeline.md
@@ -1,11 +1,27 @@
# ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

-- Status: Proposed
+- Status: Accepted and implemented (F1-F7 merged)
- Date: 2026-07-06
- Decision owner: project owner
- Tracking issue: #31 (Track F of the provider-neutral umbrella, #25)
- Depends on: ADR 0011 (Track B ? domain schema, `policies` table)

### Implementation status
+
+This ADR is implemented through the F1-F7 PR sequence tracked by umbrella
+issue #42:
+
+- #50 ? policy engine prototype and config persistence.
+- #51 ? topic preferences and caption capture.
+- #52 ? caption safety.
+- #53 ? URL/domain reputation.
+- #54 ? deep-inspection budget boundary.
+- #55 ? `policy_decisions` audit table.
+- #57 ? `/why <content_item_id>` explainability command.
+
+Follow-up retention cleanup for `policy_decisions` is tracked separately in
+issue #56.
+

```

Context

The system today evaluates every matched video against a single `FilterConfig`
@@ -25,9 +41,9 @@ them, and defer expensive work until after cheap rejections.
| `FilterConfig` / `Policy` | One JSON blob per account. Flat duration/resolution/size/exclude
rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config\_json` is the `FilterConfig`
blob. |
-| Config persistence shims | `load\_filter\_config` / `save\_filter\_config` parse and write only
the flat `FilterConfig` shape. Unknown future keys are not preserved by this path. |
-| `content\_items.text` / `link\_url` | Columns exist from ADR 0011, but the live Telegram path
does not populate captions or extracted links yet. |
-| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were
deferred to v2 in the original plan. |
+| Config persistence shims | `load\_filter\_config` / `save\_filter\_config` merge flat
`FilterConfig` updates into the default policy JSON so unknown policy keys survive legacy `/set`
commands. |
+| `content\_items.text` / `link\_url` | The Telegram path captures caption text in
`content\_items.text`. `link\_url` remains available for future link/article content; URL
reputation currently extracts URLs from captured text at evaluation time. |
+| Security / topic / deep-inspect | Topic preferences, caption safety, URL/domain reputation,
and the deep-inspection budget boundary are implemented. No production deep-inspection fetcher
is wired by default. |

What we want

@@ -214,30 +230,23 @@ keys inside that blob:

- `policies.config\_json` gains optional `topics`, `security\_checks`, and
`deep\_inspect` keys.
- The legacy `load\_filter\_config` / `save\_filter\_config` shim is not a safe
- persistence path for those keys. It parses through `FilterConfig` and then
- serializes `FilterConfig` again, so unknown keys would be dropped. F1 must
- add a policy-config persistence path that reads/writes the raw JSON object
- (or a typed superset model) and preserves unknown keys when legacy `/set`
- commands update flat `FilterConfig` fields.
- Future `policy\_decisions` table (F6) stores the explanation
- trace for debugging: `(content_item_id, policy_id, stage_name, passed,
- reason, metadata_json, evaluated_at)`.
-
- ### Implementation phasing (future issues)
-
- | Phase | Scope | Dependencies |
-|---|---|---|
-| **F1** ? Policy engine prototype | `PolicyStage` protocol, `evaluate\_pipeline()`, raw/typed
policy-config persistence, and hot-path wiring from live handler to
`ContentItem`/`Source`/`Policy` context | Tracks B-E landed |
-| **F2** ? Topic preferences | `topics` config parsing, source title matching, caption capture
into `content\_items.text`, caption keyword matching | F1 |
-| **F3** ? Caption safety | Unsafe caption term/pattern scanning from captured text | F1, F2
caption capture |
-| **F4** ? URL/domain reputation | Caption link extraction into `content\_items.link\_url` or
stage metadata, domain blacklist checks with no page fetch | F1, F2 caption capture |
-| **F5** ? Deep inspection | `ContentFetcher`, ffprobe/perceptual hash, download budget
enforcement | F1, accepted ADR 0003 amendment above |
-| **F6** ? Policy decisions table | Persistent audit trail storage for accepted and rejected
items | F1 |
-| **F7** ? Explainability endpoint | `/why <content\_item\_id>` command showing the explanation
trace | F6 |
-
- Each phase is a separate, mergeable PR ? none are bundled here. This ADR
-only defines the **design contract** the phases code against.
- + - The legacy `load\_filter\_config` / `save\_filter\_config` shim now preserves
+ unknown policy keys when legacy `/set` commands update flat `FilterConfig`

```

+ fields.
+- `policy_decisions` stores one row per `(content_item_id, policy_id)` with
+ final pass/fail, ordered stage trace JSON, and `evaluated_at`.
+
+### Implementation phasing
+
+| Phase | Scope | PR | Status |
+|---|---|---|---|
+| **F1** ? Policy engine prototype | `PolicyStage`, `evaluate_pipeline()`, config preservation,
live hot-path wiring | #50 | Merged |
+| **F2** ? Topic preferences | `topics` parsing, source-title/caption matching, caption capture
| #51 | Merged |
+| **F3** ? Caption safety | Unsafe caption term/pattern scanning | #52 | Merged |
+| **F4** ? URL/domain reputation | Captured-text URL extraction and domain blocklist checks
with no page fetch | #53 | Merged |
+| **F5** ? Deep inspection | `ContentFetcher` protocol and budget boundary | #54 | Merged |
+| **F6** ? Policy decisions table | Persistent audit trail storage for accepted and rejected
policy-stage decisions | #55 | Merged |
+| **F7** ? Explainability endpoint | `/why <content_item_id>` command showing stored traces and
safe fallback output | #57 | Merged |

```

Consequences

@@ -265,8 +274,9 @@ only defines the **design contract** the phases code against.

- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Caption and link stages require ingestion changes because the live path does not currently write `content\_items.text` or `content\_items.link\_url`.
- +- Caption capture is now wired through `content\_items.text`; URL reputation extracts links from captured text. `content\_items.link\_url` remains reserved for future link/article content.
- Deep inspection amends ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

23. assistant (2026-07-07T03:36:11.434Z)

The ADR diffs are accurate. Now let me check the runbook and plan diffs, which are large. Let me focus on concrete claims (commands, deploy steps, defaults) that could be wrong.

24. user (2026-07-07T03:36:13.629Z)

```
diff --git a/docs/runbook.md b/docs/runbook.md
```

```
index b2e3ad7..030c318 100644
```

```
--- a/docs/runbook.md
```

```
+++ b/docs/runbook.md
```

```
@@ -1,73 +1,116 @@
```

```
# Runbook: Telegram Video Filter Userbot
```

```
-Operational guide for deploying, onboarding users, and handling common issues.
-See [design.md](./design.md) for architecture and [plan.md](./plan.md) for
-build status.
```

```
+Operational guide for deployment, onboarding, source control, filtering,
+backfill, explainability, and troubleshooting.
```

1. Prerequisites

- A Telegram account you control.
- `api\_id` and `api\_hash` from <https://my.telegram.org> ? *API development tools*. Create an app (any name/desc) to get these.
- Docker + Docker Compose (recommended), **or** Python 3.11+.
- A persistent directory mounted at `/data` (holds the SQLite DB + session files ? treat as a credential).
- **Optional but recommended**: install `cryptg` (`pip install cryptg`) for native crypto acceleration. Without it Telethon falls back to slower

```

- pure-Python encryption (you'll see a warning at startup). For Docker, add
- `cryptg` to the image.
- > **_Note:_** `cryptg` is included as a project dependency by default
- > (`pyproject.toml`); you should already have it installed.
-
-## 2. First-time deploy (Docker)
+- `api_id` and `api_hash` from <https://my.telegram.org>.
+- Docker + Docker Compose, or Python 3.11+.
+- A persistent `data/` directory for SQLite and Telethon session files.
+- Optional local virtualenv for scripts/tests:
+
+```powershell
+python -m venv .venv
+.\venv\Scripts\python.exe -m pip install -e .[dev]
+```
+
+`cryptg` is a runtime dependency and should already be installed through the
+project package.
+
+## 2. First Deploy
+
+```bash
+# 1. clone
+git clone https://github.com/avidullu/telegram-video-filter.git
+cd telegram-video-filter
+
+# 2. configure env
+cp .env.example .env
+# edit .env: set TG_API_ID, TG_API_HASH, DB_PATH=/data/app.db
+
+# 3. (optional) pre-create the data volume
+mkdir -p data
+# edit .env: TG_API_ID, TG_API_HASH, DB_PATH, SESSIONS_DIR
+
+# 4. build + run in background
+docker compose up -d --build
+docker compose logs -f app
+```
+
+-Expected startup log: `manager_started {users: 0}` until you onboard a user.
+Expected first log state: the manager starts with no runnable sessions until a
+user is bootstrapped.
+
+## 3. Onboard a user (interactive, once per user)
+## 3. Onboard a Telegram User Session
+
+-v1 onboards via a host-side CLI that performs the interactive Telegram login and
+-writes a session file into `data/sessions/`.
+Run the bootstrap command in the same environment that can write to `data/`:
+
+```bash
+# from the repo root, inside the same env/container that runs the app
+python -m tg_video_filter.bootstrap
+# ? prints the banner (session dir + db path)
+# ? prompts: phone (E.164), then the login code Telegram sends you,
+#           then 2FA password if enabled.
+# ? creates data/sessions/user_{id}.session
+# ? inserts a users row (enabled=1) + default filter_config
+```
+
+-After onboarding, restart the container so the manager picks up the new user:
+It prompts for:
+
+1. phone number,
+2. Telegram login code,

```

```

+3. 2FA password if enabled.
+
+It then creates:
+
+- `accounts` row,
+- `telegram_sessions` row,
+- legacy-compatible `users` view data,
+- default policy/config rows,
+- `data/sessions/user_{id}.session`.
+
+Restart the manager after onboarding:

```bash
docker compose restart app
-# ? logs: client_connected {user_id: <id>}
```

-> ?? **The session file is a credential.** Anyone with `user_{id}.session` can
-> act as that Telegram account. Keep `data/` private and backed up.
+Session files are credentials. Back up and protect `data/`.
+
+## 4. Live Source Scanning
+
+Live ingestion is opt-in per source.
+
+Unknown sources are lazily discovered as `watched=false`; they are listed but
+not ingested until you watch them. Send commands to your own Saved Messages:
+
+```text
+/channels
+/watch <id|@name|title-fragment>
+/unwatch <id|@name|title-fragment>
+/watchlist
+```
+
+Typical flow:
+
+1. Let new traffic arrive, or run `/backfill` to discover historical dialogs.
+2. Run `/channels`.
+3. Run `/watch -1001234567890` or `/watch Some Channel`.
+4. Confirm with `/watchlist`.
+
+## 5. Filter Commands
+
+These commands persist config changes and hot-reload the in-memory filter
+engine; no restart is needed for these fields.
+
+```text
+/show config
+/show commands
+/set size 10MB
+/set length 15m
+/set max length 15m
+/set min length 5s
+/set min resolution 720p
+/set min res 720p
+/set exclude gif on
+/set exclude round off
+/set exclude nosound on
+```
+
+Units:

-## 4. Adjusting filters
+- size: `KB`, `MB`, `GB`, `TB`; default is MB.

```

+ duration: `s`, `m`, `h`; default is seconds.

-Filters are stored per-user as JSON in `filter\_config`. Default v1 config:

+Default flat filter config for fresh installs:

```
```json
{
@@ -75,224 +118,219 @@ Filters are stored per-user as JSON in `filter_config`. Default v1
config:
 "max_duration_s": 7200,
 "target_height": 1080,
 "max_size_mb": 2048,
+ "exclude_gif": true,
+ "exclude_round": true,
+ "exclude_nosound": false,
 "delivery_mode": "forward",
 "delivery_target": "me"
}
```
```

-To change a user's config (no v1 UI yet ? use sqlite3 directly):

+### 6. Delivery Mode and Target

```
-```bash
-sqlite3 data/app.db
-# find the user
-SELECT id, phone FROM users;
-# update (example: allow up to 4h, drop min duration)
-UPDATE filter_config
-  SET config_json = json('{"min_duration_s":0,"max_duration_s":14400,"target_height":1080,"max
_size_mb":2048,"delivery_mode":"forward"}')
-  WHERE user_id = <id>;
-.quit
-docker compose restart app
-```
```

+Delivery mode and target are stored in policy/config JSON. There is no

+Saved-Messages command for changing them yet, so update deliberately and restart

+the manager.

-### Backfill historical videos (v1.1)

+Modes:

-> ?? Backfill scans **all** channels for historical videos. On large channels

-> it can run for minutes and may trigger Telegram rate limits. Progress is

-> reported in real time to your Saved Messages.

+| Mode | Behavior |

+|---|---

+| `forward` | Forward the Telegram video message. |

+| `link` | Send a `t.me` link plus metadata summary; falls back to forward if no link exists. |

+| `both` | Forward and send the link summary. |

-While the manager is running, send a message to your **Saved Messages**:

+Targets:

+| Target | Meaning |

+|---|---

+| `me` | Saved Messages. |

+| `@your-feed` | Public channel or chat username. |

+| `-1001234567890` | Numeric Telegram chat/channel id. |

+

+Use the helper script for targets:

+

+```powershell

+.\venv\Scripts\python.exe scripts\set_delivery_target.py <account_id> <target>

```

```

-/backfill # scan last 90 days (default)
-/backfill 7d # scan last 7 days
-/backfill 8h # scan last 8 hours
-/backfill 2w # scan last 2 weeks
-/backfill 30 # scan last 30 days
-```

-The bot responds:
+If Telethon cannot resolve a target, refresh the entity cache:
+
+```powershell
+.\.venv\Scripts\python.exe scripts\resolve_target.py <account_id> <target>
+```

-? Starting backfill of all channels (last 90 days)...
-? Scanning #ChannelName (last 90 days)...
- #ChannelName: 142 videos, 8 matched so far...
-? #ChannelName: 156 videos, 11 matched
-? Backfill complete: 5 channels scanned, 823 videos found, 47 matched
+
+Then restart:
+
+```bash
+docker compose restart app
+```

-- Videos already seen (by live watcher or prior backfill) are skipped.
-- Matches are delivered according to your current `delivery_mode`.
-- If a channel triggers a flood wait, it sleeps and continues.
+Backfill progress messages always go to Saved Messages; delivered matches honor
+`delivery_target`.

-### Tuning filter criteria live (v1.3)
+## 7. Backfill

-While the manager is running, send these commands to Saved Messages:
+Backfill scans historical channel/group dialogs and can be expensive on large
+accounts. It does not use the live `watched` gate; run it intentionally.

-```
-/show config # display current filter settings
-/show commands # list all available commands
-/set size 10MB # set max file size
-/set length 15m # set max video duration
-/set max length 15m # same as above (alias)
-/set min length 5s # set min video duration
-/set exclude gif on # exclude Telegram GIFs
-/set exclude round on # exclude circular video messages
-/set exclude nosound on # exclude videos without audio
+```text
+/backfill
+/backfill 8h
+/backfill 7d
+/backfill 2w
+/stop
+```

-All changes persist to the DB and take effect immediately ? no restart needed.
-Exclusions are off by default for backwards compatibility.
+Behavior:

-### Switching delivery mode (v1.1)
+- default window is 90 days;
+- dedup skips already-seen content;
+- matching content is delivered with current delivery mode/target;
+- progress is reported to Saved Messages;

```



+ - flood waits are slept through and logged.

-The ``delivery\_mode`` field controls how matched videos are delivered to your

-Saved Messages:

### 8. Policy Pipeline

-| Mode | What you get |

-|---|---|

-| ``forward`` (default) | The video itself is forwarded (re-uploaded). |

-| ``link`` | A text message with a ``t.me`` link + metadata summary (resolution, duration, size). No re-upload. |

-| ``both`` | Forward the video AND send the link message. |

+Metadata filters run first. If they pass, ADR 0012 policy stages run in order:

-Example ? switch to link-only mode:

+1. topic preferences,

+2. URL/domain reputation,

+3. caption safety,

+4. deep inspection boundary.

```sql

-UPDATE filter_config

- SET config_json = json_set(config_json, '\$.delivery_mode', 'link')

- WHERE user_id = <id>;

```

+Flat ``/set`` commands update ``FilterConfig`` fields inside the default policy

+without erasing unknown policy keys.

-Then restart the manager. Old configs without ``delivery\_mode`` default to

-``forward`` (backwards-compatible).

+Policy-stage config currently has no friendly command UI. Edit

+``policies.config\_json`` carefully when needed. Example:

### Routing matches to a channel you own (v1.2)

-

-By default matches go to your ``Saved Messages``. Set ``delivery\_target`` to

-route them to a channel instead (see ADR 0005):

```sql

+UPDATE policies

+ SET config_json = json_set(

+ config_json,

+ '\$.topics',

+ json('{"include":["ai"],"exclude":["crypto"],"mode":"any"}')

+)

+ WHERE account_id = <account_id>

+ AND is_default = 1;

```

-| Value | Meaning |

-|---|---|

-| ``me`` (default) | Saved Messages. |

-| ``@your-feed`` | A public channel the account is a member of. |

-| ``-1001234567890`` | A private channel's chat id (the ``-100?`` form). |

+Caption and domain checks:

```sql

-UPDATE filter_config

- SET config_json = json_set(config_json, '\$.delivery_target', '@my-movie-feed')

- WHERE user_id = <id>;

+UPDATE policies

+ SET config_json = json_set(

+ config_json,

+ '\$.security_checks',

+ json('{

+ "blocked_domains": ["bad.example"],

```
+         "unsafe_caption_terms": ["malware"],
+         "unsafe_caption_patterns": ["free\\\\s+money"]
+     })
+ )
+ WHERE account_id = <account_id>
+ AND is_default = 1;
+ 
```

-Then restart the manager. Requirements:

+Keep `deep\_inspect` disabled unless a production fetcher/budget is explicitly wired:

```
+
+```json
+{ "deep_inspect": false }
+`
```

```
-- The userbot account must be a member of the target channel and have
- post rights (admin) if posting is restricted.
-- Backfill progress messages still go to Saved Messages ? only matched
- videos/links honor `delivery_target`.
-- Old configs without `delivery_target` default to `me` (backwards-compatible).
+## 9. Explain Decisions
```

```
-Instead of hand-writing SQL, use the helper script (run with the venv python,
-not the bare `python` on your PATH ? the project's deps live in .venv):
+Use `/why <content_item_id>` from Saved Messages.
```

```
- # Windows (PowerShell)
- .\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py <user_id> <target>
- # macOS / Linux
- .venv/bin/python scripts/set_delivery_target.py <user_id> <target>
+Find recent content item ids:
```

```
-#### Troubleshooting: "Cannot find any entity corresponding to ..."
+```sql
+SELECT id, provider, provider_message_id, matched, seen_at
+ FROM content_items
+ ORDER BY id DESC
+ LIMIT 20;
+`
```

```
-If delivery fails with `Cannot find any entity corresponding to "-100..."`,
-Telethon's session cache has never recorded that channel's `access_hash`.
-Resolve and cache it with:
+`/why` behavior:
```

```
- .\\.venv\\Scripts\\python.exe scripts\\resolve_target.py <user_id> <target>
+- reads only content owned by the requesting account;
+- renders persisted `policy_decisions` traces when present;
+- falls back to recorded item status plus current metadata-filter replay when
+ no policy decision exists;
+- escapes untrusted captions/URLs/metadata and caps the reply under Telegram's
+ message limit.
```

```
-This stops the manager (the session file can't be shared), refreshes the
-entity cache via `get_dialogs()`, verifies the target resolves, and writes
-the cached entity back to the session file. Then restart the manager.
+The fallback replay uses current config and persisted metadata. It can differ
+from the original decision if config changed, and GIF/round/nosound flags are
+not persisted for replay.
```

```
-## 4b. How video deduplication works (v1.3)
+## 10. Deduplication
```

-The userbot will not forward the same video twice thanks to a **three-tier**

```

-dedup** pipeline (checked cheapest-first; see ADR 0006):
+Dedup is automatic and metadata-first:

Tier	Key	Catches
-	**0**	`(user_id, channel_id, message_id)`
-| **1** | `(user_id, document_id)` | The same Telegram file **forwarded** across channels ?
Telegram reuses `document.id`. DB-enforced via a partial unique index. |
-| **2** | `(size_bytes, duration_s, height, width)` | A re-upload of the identical file (new
`document.id`, same bytes). Heuristic lookup, no DB constraint. |
-
-This is fully automatic and adds no bandwidth cost ? it relies only on metadata
-already present in the message. A re-encode (different resolution/codec/size)
-is **intentionally not deduped** ? it is a different file and is evaluated
-independently by the filter engine.
+| 0 | source/message | Same Telegram message instance. |
+| 1 | Telegram `document.id` | Same file forwarded across channels. |
+| 2 | size/duration/height/width | Re-upload of identical media metadata. |

-Database migration: existing v1 databases are upgraded automatically on the
-next `init_db()` call via a `PRAGMA user_version` migration ladder. The
-`document_id` column is added with `ALTER TABLE` (nullable; existing rows get
-`NULL`) and the partial unique index is created. The migration is idempotent
-and safe to re-run. No manual SQL is needed.
+Re-encodes are intentionally treated as distinct content.

-## 5. Rotate a user session
+## 11. Backups

-Use when a session is suspected compromised, a phone number changes, or login
-starts failing.
+The important state is `data/`:

```bash
docker compose stop app
rm data/sessions/user_{id}.session # remove old session
-# also kill it from Telegram side: Settings ? Devices ? terminate the session
-python -m tg_video_filter.bootstrap # re-login interactively
+tar -czf backup-$(date +%F).tar.gz data/
docker compose start app
```

```

25. assistant (2026-07-07T03:36:17.438Z)

The runbook adds references to helper scripts `scripts/set\_delivery\_target.py` and `scripts/resolve\_target.py`. Let me verify these scripts exist (they were referenced in old runbook too, but the new one adds `resolve\_target.py` for a new purpose). Also verify the `why` fallback claim about GIF/round/nosound flags not persisted for replay, and let me check the plan.md diff briefly plus the rest of runbook.

26. user (2026-07-07T03:36:21.362Z)

```

resolve_target.py
set_delivery_target.py
---plan---
+4. Opt in live ingestion with `/watch`.
+5. Deduplicate content into `content_items`.
+6. Evaluate metadata filters and ADR 0012 policy stages.
+8. Deliver accepted content by forward, link, or both.
+| Telethon ingestion | Complete | Metadata extraction, source gate, dedup, policy evaluation,
persistence, delivery. |
+| Delivery | Complete | `forward`, `link`, `both`; configurable target; delivery attempts. |
+| Bootstrap | Complete | Interactive login creates session file and default
account/session/config rows. |

```

```

+| Inline commands | Complete | `/show`, `/set`, source watch commands, `/why`. |
+| Content dedup | Complete | Message instance, Telegram `document.id`, metadata fingerprint. |
+| Selective scanning | Complete | `sources.watched` and Saved Messages watchlist commands. |
+| Policy pipeline | Complete | ADR 0012 F1-F7: engine, topics, caption safety, URL reputation,
deep-inspection boundary, audit table, `/why`. |
+| F3 - Caption safety stage | #45 | #52 | Merged |
+| F4 - URL/domain reputation stage | #46 | #53 | Merged |
+| F5 - Deep inspection fetcher + budget boundary | #47 | #54 | Merged |
+| F6 - Policy decisions audit table | #48 | #55 | Merged |
+Umbrella issue #42 is complete. Policy-decision retention cleanup remains open
+- `/set size <N><unit>`
+- `/set length <N><unit>` or `/set max length <N><unit>`
+- `/set min length <N><unit>`
+- `/set min resolution <N>p` or `/set min res <N>p`
+- `/set exclude gif|round|nosound on|off`
+- `/watch <id>@<name>|<title>-fragment`
+- `/watchlist`
+- Production deep-inspection fetcher/downloader.
+- Policy-decision retention cleanup (#56).
+The original v1 plan was a lean userbot that watched every subscribed channel
+and forwarded matching videos to Saved Messages. That was useful early, but the

```

27. assistant (2026-07-07T03:36:25.438Z)

Both helper scripts exist. The plan.md claims look accurate. Let me verify the one nuanced claim in runbook \$9: "GIF/round/nosound flags are not persisted for replay." Let me check the `/why` fallback in command_service and whether content_items persists those flags. Let me look at the `/why` handler and get_content_item_for_account and the fallback replay.

28. user (2026-07-07T03:36:26.470Z)

```

525         async def _handle_why(
526             self,
527             conn: aiosqlite.Connection,
528             user_id: int,
529             item_id_text: str | None,
530             current_cfg: FilterConfig,
531         ) -> CommandResult:
532             """Explain a recorded content item's policy decision."""
533             if item_id_text is None or not item_id_text.strip().isdecimal():
534                 return CommandResult("? Usage: `/why <content_item_id>`")
535             item_id = int(item_id_text.strip())
536             if item_id < 1:
537                 return CommandResult("? Usage: `/why <content_item_id>`")
538
539             item = await db.get_content_item_for_account(conn, user_id, item_id)
540             if item is None:
541                 return CommandResult(f"? Content item `{item_id}` was not found for your
account.")
542
543             assert item.id is not None
544             decisions = await db.list_policy_decisions_for_content(conn, item.id)
545             if decisions:
546                 return CommandResult(_format_policy_decisions(item, decisions))
547             return CommandResult(_format_missing_policy_decision(item, current_cfg))
548
549         async def _resolve_source(
550             self, conn: aiosqlite.Connection, user_id: int, target: str
551         ) -> Source | None:
552             """Resolve a user-provided source identifier to a ``Source`` row.
553
554             Accepts numeric channel id, ``@username``, or channel title.
555             ``@`` prefix is stripped before lookup.
556
557             """
558             target = target.removeprefix("@")

```

```

558         # Try by provider-native numeric id (the common case).
559         src = await db.get_source_by_provider_id(conn, user_id, "telegram", target)
560         if src is not None:
561             return src
562         # Try by @username.
563         src = await db.get_source_by_provider_id(conn, user_id, "telegram",
f"@{target}")
564         if src is not None:
565             return src
566         # Try by title (case-insensitive substring match).
567         all_srcs = await db.list_account_sources(conn, user_id)
568         target_lower = target.lower()
569         for s in all_srcs:
570             if s.title and target_lower in s.title.lower():
571                 return s
572         return None
573
574     # -- internal -----
575
576     async def _update(
577         self,
578         conn: aiosqlite.Connection,
579         user_id: int,
580         current: FilterConfig,
581         max_size_mb: int | None = None,
582         max_duration_s: int | None = None,
583         min_duration_s: int | None = None,
584         target_height: int | None = None,
585         exclude_gif: bool | None = None,
586         exclude_round: bool | None = None,
587         exclude_nosound: bool | None = None,
588     ) -> FilterConfig:
589         """Apply partial updates to the account's config, persist, and return."""
590         updates: dict[str, object] = {}
591         if max_size_mb is not None:
592             updates["max_size_mb"] = max_size_mb
593         if max_duration_s is not None:
594             updates["max_duration_s"] = max_duration_s
595         if min_duration_s is not None:
596             updates["min_duration_s"] = min_duration_s
597         if target_height is not None:
598             updates["target_height"] = target_height
599         if exclude_gif is not None:
600             updates["exclude_gif"] = exclude_gif
601         if exclude_round is not None:
602             updates["exclude_round"] = exclude_round
603         if exclude_nosound is not None:
604             updates["exclude_nosound"] = exclude_nosound
605         cfg = current.model_copy(update=updates) if updates else current
606         await db.save_filter_config(conn, user_id, cfg)
607         log.info(
608             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
609             "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
610             user_id,
611             cfg.max_size_mb,
612             cfg.min_duration_s,
613             cfg.max_duration_s,
614             cfg.target_height,
615             cfg.exclude_gif,
616             cfg.exclude_round,
617             cfg.exclude_nosound,
618         )
619         return cfg
620
621

```

```

622     __all__ = [
623         "ENGINE_KEY",
624         "CommandResult",
625         "CommandService",
626         "_WHY_RE",
627         "_format_duration",
628         "_format_size",
629         "format_config",
630         "format_commands",
631         "parse_duration_s",
632         "parse_size_mb",
633     ]
634

```

29. assistant (2026-07-07T03:36:32.594Z)

The ``why`` handler confirms account-scoping (``get_content_item_for_account``), stored trace rendering, and fallback. The `ContentItem` model has no exclude flags persisted (confirmed in `models.py` ? only duration/width/height/size/mime), so the runbook §9 claim about GIF/round/nosound flags not being persisted for replay is accurate.

Now let me verify one remaining thing in the README §Quick Start: it shows ``docker compose up -d --build`` THEN ``python -m tg_video_filter.bootstrap`` THEN restart. This is a plausible ordering. Let me double check the design.md observability event names against actual log calls, since those are concrete claims. Design lists: ``video_forwarded``, ``delivery_record_failed``, ``config_updated``, ``flood_wait``, ``client_connected``, ``client_disconnected``, ``manager_started``, ``video_seen``.

30. user (2026-07-07T03:36:34.913Z)

```

C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\delivery.py:111:
log.warning("flood_wait video=%s seconds=%s retrying_after=%.1f", video_id, e.seconds, wait)
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\delivery.py:159:
log.info("video_forwarded video=%s target=%s", video_id, target)
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\delivery.py:273:
"link_mode_fallback_to_forward video=%s mode=%s link_text_missing=%s",
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\manager.py:73:
log.info("manager_started sessions=%d db=%s", len(pairs), settings.db_path)
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\manager.py:177:
log.info("client_connected account=%s", session.account_id)
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\manager.py:181:
log.info("client_disconnected account=%s", session.account_id)
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\command_service.py:608:
"config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\telethon_client.py:335:
"video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",

```

31. assistant (2026-07-07T03:36:37.295Z)

All log event names in design.md §10 are present in code except ``delivery_record_failed``. Let me verify that one and confirm the exact string.

32. user (2026-07-07T03:36:38.606Z)

No matches found

33. assistant (2026-07-07T03:36:40.978Z)

``delivery_record_failed`` does not exist in the codebase. Let me check what delivery-record failure log actually exists, to confirm this is a wrong/invented event name.

34. user (2026-07-07T03:36:42.499Z)

```

111:         log.warning("flood_wait video=%s seconds=%s retrying_after=%.1f", video_id,

```

```
e.seconds, wait)
159:         log.info("video_forwarded video=%s target=%s", video_id, target)
191:         log.info("link_sent video=%s target=%s", video_id, target)
266:         log.warning("link_delivery_failed_in_both_mode video=%s", video_id)
268:         log.warning("forward_delivery_failed_in_both_mode video=%s", video_id)
```

35. assistant (2026-07-07T03:36:45.272Z)

Let me check the full delivery.py for any record-failure log and telethon_client, to be thorough before flagging `delivery\_record\_failed` as nonexistent.

36. user (2026-07-07T03:36:46.819Z)

```
275:         log.warning(
```

37. user (2026-07-07T03:36:49.321Z)

```
270             Channel(user_id=user.id, channel_id=channel_id,
title=_channel_title(event)),
271         )
272         if channel.id is not None:
273             source = await db.get_source_by_id(conn, channel.id)
274         except Exception:
275             log.warning(
276                 "failed to upsert channel %s for user %s", channel_id, user.id,
exc_info=True
277         )
278
279         text = _message_text(message)
280         result = engine_holder[ENGINE_KEY].evaluate(meta)
281         matched = result.matched
```

38. assistant (2026-07-07T03:36:54.281Z)

Confirmed: there is no `delivery\_record\_failed` log event anywhere in the codebase. This is a low-severity accuracy issue in the design.md observability list (an invented event name an operator might grep for). The actual delivery-failure events are `link\_delivery\_failed\_in\_both\_mode` / `forward\_delivery\_failed\_in\_both\_mode`.

This is minor. Let me do a final check on one more concrete claim worth verifying: the README/design say `/set exclude gif|round|nosound on|off` and defaults "GIF/round excluded, nosound allowed" ? matches models.py (exclude_gif=True, exclude_round=True, exclude_nosound=False). Correct.

Also the README Development section says "80% coverage gate." Let me verify against pyproject.

39. user (2026-07-07T03:36:55.644Z)

```
111:# Final gate: 80%. Interactive/Telethon-glue entrypoints are marked
113:fail_under = 80
```

40. assistant (2026-07-07T03:37:00.539Z)

80% coverage gate confirmed.

I've now cross-checked every concrete claim in README.md and docs/design.md (plus the supporting ADR/runbook/plan/source-edit claims) against the merged code. The documentation is remarkably accurate. Every command, table name, default value, dedup tier, delivery mode, policy-stage order, opt-in scanning description, and "what is not built" entry matches the code. The only factual defect I found is one invented log-event name in the design.md observability list.

Let me compile the findings.

41. user (2026-07-07T03:37:08.178Z)

Structured output provided successfully